

```

/* ===== 微信小游戏版 minigame/js/core.js ===== */
/* eslint-disable */
/* 核心逻辑：由 extract_core.py 从网页版 index.html 原样抽出，未修改任何算法。
   这一层不碰 DOM、不碰 wx API，浏览器与小游戏共用。
   网页版算法有改动时重跑 extract_core.py 同步，不要手改本文件。 */

/* levelDef 按模式取配置，网页版直接读全局 G。这里保留原样，
   由游戏层在调用 genLevel 之前写 CORE.G，把耦合摆在明面上。 */
var G = { mode: "normal", wave: 1, buffs: null, matches: 0,
          tiles: [], tray: [], rain: [], goal: null, rcfg: null, lv: 1,
          toolLv: { shuffle: 1, undo: 1, out: 1 } };

/* 有解洗牌里会查道具等级，网页版从 save 里读。这里读 G，语义一致。 */
function toolLv(k) { return (G.toolLv && G.toolLv[k]) || 1; }
function mulberry32(a){
  return function(){
    a|=0; a=a+0x6D2B79F5|0;
    let t=Math.imul(a^a>>>15,1|a);
    t=t+Math.imul(t^t>>>7,61|t)^t;
    return ((t^t>>>14)>>>0)/4294967296;
  };
}

let RNG=Math.random;

function seedRNG(seed){ RNG = (seed==null) ? Math.random : mulberry32(seed>>>0); }

const rnd = n => Math.floor(RNG()*n);

function shuffle(a){for(let i=a.length-1;i>0;i--){const j=rnd(i+1);[a[i],a[j]]=[a[j],a[i]]};return a;}

const LV_CAP=200;

const LEVELS=[
  {name:'晨露坡', pos:[12,6], cfg:{n:18, layers:3, types:4, q:[], d:3}},
  {name:'微光林', pos:[55,14], cfg:{n:24, layers:3, types:4, q:[], d:4}},
  {name:'风车草地', pos:[20,22], cfg:{n:30, layers:4, types:5, q:[6], d:4}},
  {name:'薄荷溪', pos:[60,30], cfg:{n:36, layers:4, types:6, q:[6], d:3}},
  {name:'云杉谷', pos:[14,38], cfg:{n:42, layers:5, types:6, q:[6], d:3}},
  {name:'阳光丘', pos:[58,46], cfg:{n:45, layers:5, types:6, q:[6,6], d:5}},
  {name:'苔藓洞', pos:[18,54], cfg:{n:48, layers:5, types:6, q:[9,9], d:6}},
  {name:'松果径', pos:[56,62], cfg:{n:54, layers:6, types:6.4, q:[9,9], d:4}},
  {name:'野花甸', pos:[16,70], cfg:{n:60, layers:6, types:7, q:[9,9], d:3}},
  {name:'远山径', pos:[58,78], cfg:{n:66, layers:6, types:7, q:[12,12], d:5}},
  {name:'星月原', pos:[20,86], cfg:{n:72, layers:7, types:7, q:[12,12], d:6}},
  {name:'天空牧场', pos:[55,94], cfg:{n:81, layers:7, types:7.2, q:[12,12], d:4}}
];

const MAX_LEVEL=LV_CAP;

const CH_SIZE=10;

const CHAPTERS=[
  {name:'晨露草原', cls:'ch-dawn', ico:'fa-mountain-sun',
    intro:'天刚亮，露水还挂在草尖上。羊群三三两两地散在坡上，谁也不着急。 | 牧羊人说，这片坡是最好学的地方——摔一跤也不疼。',
    words:['晨露坡','微光林','风车草地','薄荷溪','云杉谷','阳光丘','苔藓洞','松果径','野花甸','远山径']},
  {name:'幽深森林', cls:'ch-forest', ico:'fa-tree',

```

```

    intro:'越往里走,树越密,光越少。有些草料被藤蔓缠住了,得先解开才能拿。 | 林子里的羊学会了一件事:看不清的时候,先别急着动手。',
    words:['星月原','天空牧场','低语林','菌菇径','鹿鸣谷','藤蔓崖','萤火沼','古木道','静水潭','暮色岭']],
    {name:'初雪高原',cls:'ch-snow',ico:'fa-snowflake',
    intro:'第一场雪落下来的时候,整片高原都安静了。冻住的草料要靠旁边的温度化开。 | 这里的规矩是:慢一点没关系,但别停下。',
    words:['初雪坡','冰棱谷','白桦道','霜风口','雪兔径','冻湖畔','银松林','寒星台','雪线崖','极光原']},
    {name:'云海之巅',cls:'ch-cloud',ico:'fa-cloud-sun',
    intro:'再往上就出了云层。脚下是翻涌的云海,头顶是从没见过这么近的太阳。 | 这里风大,草却长得格外好 — 牧羊人说,越难站住的地方,越值得多待一会儿。',
    words:['入云口','金光坡','浮岛渡','霞光谷','风起崖','云中湖','日照原','断云桥','天梯道','云海之巅']},
    {name:'星夜牧场',cls:'ch-night',ico:'fa-moon',
    intro:'再往上就是星夜牧场了。据说走到这里的羊,都能在天上找到属于自己的那一颗。 | 牧羊人没有再说什么,只是把灯点亮,站在原地看着羊群往前走。',
    words:['夜幕坡','银河渡','月牙湾','流星径','梦境园','星尘谷','夜莺林','幻光崖','无垠原','天际牧场']}
];

const ROMAN=['','I','II','III','IV','V','VI','VII','VIII','IX'];

function chapterIdx(lv){ return Math.floor((lv-1)/CH_SIZE); }

function chapterOf(lv){ return CHAPTERS[chapterIdx(lv)%CHAPTERS.length]; }

function chapterTitle(lv){
    const round=Math.floor(chapterIdx(lv)/CHAPTERS.length)+1;
    return chapterOf(lv).name+(round>1?' '+ROMAN[Math.min(round,9)]:'');
}

function isBoss(lv){ return lv%CH_SIZE===0; }

const GOALS=['clear','steps','collect','peel','rescue','sprint'];

function goalOf(lv){
    if(lv<=12||isBoss(lv)||isBonus(lv)) return 'clear'; /* 前 12 关与 Boss/宝箱保持清空 */
    return GOALS[(lv-13)%GOALS.length];
}

function makeGoal(kind,tiles,order,stats){
    const total=stats.groups;
    const idxOfMatch=k=>{ /* 第 k 组消除发生在顺序的第几步 */
        for(let i=0;i<order.length;i++) if(stats.before[i]>=k) return i;
        return order.length;
    };
    if(kind==='steps'){
        const need=Math.max(6,Math.round(total*0.72));
        const budget=Math.ceil(idxOfMatch(need)*1.09)+1;
        return {t:'steps',need,budget,used:0,got:0};
    }
    if(kind==='collect'){
        const cnt={};
        tiles.forEach(t=>{ if(t.sp!=='rain') cnt[t.type]=(cnt[t.type]||0)+1; });
        const cand=Object.keys(cnt).map(Number).filter(c=>cnt[c]>=9);
        const type=cand.length?cand[rnd(cand.length)]:+Object.keys(cnt)[0];
        const need=Math.min(4,Math.max(3,Math.floor(cnt[type]/3)));
        return {t:'collect',type,need,got:0};
    }
}

```

```

if(kind==='peel'){
  const layers=Math.max.apply(null,tiles.filter(t=>t.q<0).map(t=>t.layer));
  const from=Math.max(1,layers-3);          /* 最上面四层 */
  const targets=tiles.filter(t=>t.q<0&&t.layer>=from);
  return {t:'peel',from,need:targets.length,got:0};
}
if(kind==='rescue'){
  /* 挑一张埋得最深的牌当「被困的小羊」 */
  const board=tiles.filter(t=>t.q<0);
  const deep=board.filter(t=>t.layer===Math.min.apply(null,board.map(x=>x.layer)));
  const pick=deep[rnd(deep.length)];
  if(pick) pick.rescue=true;
  return {t:'rescue',need:1,got:0};
}
if(kind==='sprint'){
  /* 模拟器测不了时间压力，所以把「非时间部分」也提上来，实际难度只会更高 */
  const need=Math.max(6,Math.round(total*0.70));
  return {t:'sprint',need,got:0,secs:60+Math.round(need*3.2)};
}
return {t:'clear',need:0,got:0};
}

function isBonus(lv){ return lv>12 && lv%CH_SIZE===5; }

const SHAPES=[
  {id:'square', n:'方阵', m:()=>true},
  {id:'diamond', n:'菱形', m:(u,v)=>Math.pow(Math.abs(u-.5),1.6)+Math.pow(Math.abs(v-.5),1.6)<
= Math.pow(.56,1.6)},
  {id:'circle', n:'圆丘', m:(u,v)=>(u-.5)*(u-.5)+(v-.5)*(v-.5)<=.30},
  {id:'tree', n:'松树', m:(u,v)=>(v>=.72 ? Math.abs(u-.5)<=.24 : Math.abs(u-.5)<=.16+v*.62)
},
  {id:'cross', n:'风车', m:(u,v)=>Math.abs(u-.5)<=.32||Math.abs(v-.5)<=.32},
  {id:'twin', n:'双峰', m:(u,v)=>{
    const d1=Math.hypot(u-.30,v-.40), d2=Math.hypot(u-.70,v-.40);
    return d1<=.38||d2<=.38||v>=.66; }},
  {id:'heart', n:'心形', m:(u,v)=>{
    const x=(u-.5)*1.85, y=(.82-v)*1.85;
    const a=x*x+y*y-.58;
    return a*a*a - x*x*y*y*.58 <= 0; }},
  {id:'tower', n:'高塔', m:(u,v)=>Math.abs(u-.5)<=.22+v*.28}
];

function shapeOf(lv){
  if(lv<=3) return SHAPES[0];          /* 前三关保持方阵，教学期不要花样 */
  const h=(lv*2654435761)>>>0;
  return SHAPES[h%SHAPES.length];
}

function spMix(lv){
  const m={};
  if(lv>=3) m.coin=.05;
  if(lv>=4) m.dbl=.10;
  if(lv>=5) m.wool=.03;
  if(lv>=6) m.rain=.09;
  if(lv>=8) m.freeze=.08;
  if(lv>=9) m.gift=.02;
  if(lv>=12) m.chain=.05;
  if(lv>=14) m.supply=.05;
  if(lv>=16) m.mystery=.07;
  if(lv>=18) m.torch=.03;

```

```

    if(lv>=22) m.bomb=.03;
    if(lv>=34) m.mystery=.09;
    if(lv>=42) m.freeze=.10;
    return m;
}

function levelDef(lv){
    if(G.mode==='endless'){
        const base=JSON.parse(JSON.stringify(rawLevelDef(waveLevel(G.wave)).cfg));
        const b=G.buffs||{};
        if(b.slots) base.slots=BASE_SLOT+b.slots;
        if(b.types) base.types=Math.max(4,+(base.types-b.types).toFixed(2));
        if(b.tiles) base.n=Math.max(18,base.n-b.tiles);
        base.shape=shapeOf(9+G.wave);
        return {name:'无尽草原',cfg:base};
    }
    if(G.mode==='weekly'){
        const base=JSON.parse(JSON.stringify(rawLevelDef(WEEKLY_LV).cfg));
        base.shape=shapeOf(WEEKLY_LV+weekIndex());
        weeklyMod().apply(base);
        return {name:'本周挑战 · '+weeklyMod().n, cfg:base};
    }
    return rawLevelDef(lv);
}

function rawLevelDef(lv){
    if(isBonus(lv)){
        /* 宝箱关: 稳赢 + 必掉图鉴, 作用是给曲线一个喘息, 不是给难度 */
        return { name:'宝箱营地', bonus:true, cfg:{
            n:36, layers:4, types:5, q:[6], d:3, shape:shapeOf(lv),
            sp:{rain:.10, coin:.14, wool:.10, gift:.06}
        }};
    }
    if(lv<=LEVELS.length){
        const L=LEVELS[lv-1];
        L.cfg.shape=shapeOf(lv);
        return L;
    }
    const t=Math.min(1,(lv-12)/38); /* 设计范围铺到第 50 关, 之后进入平台期 */
    return {
        name:chapterOf(lv).words[(lv-1)%CH_SIZE]+(chapterIdx(lv)>=CHAPTERS.length?' '+ROMAN[Math.min
(Math.floor(chapterIdx(lv)/CHAPTERS.length)+1,9)]:''),
        cfg:{
            n:Math.round((78+18*t)/3)*3,
            layers:7,
            /* d 必须随关号单调上升。之前按 4/5/6 循环, 会让曲线出现锯齿 */
            types:+(7.2+0.8*t).toFixed(2),
            q:[12,12],
            /* Boss 关的压力来源是限时, 盘面上不该再叠一层 — d 封顶 5 */
            d:(isBoss(lv)?Math.min(5,4+Math.round(1.5*t)):4+Math.round(1.5*t)),
            shape:shapeOf(lv),
            biq:lv>=14, /* 双向牌堆: 两端都能取, 是约束放宽 */
            sp:spMix(lv)
        }
    };
}

function levelName(lv){ return levelDef(lv).name; }

const TYPE_DEFS=[

```

```

    {i:'fa-leaf',      e:'🌿', c:'#4C9A5A'},
    {i:'fa-seedling',  e:'🌱', c:'#7DBB6E'},
    {i:'fa-tree',      e:'🌳', c:'#2E6B3E'},
    {i:'fa-mountain',  e:'🏔️', c:'#8FB98A'},
    {i:'fa-droplet',   e:'💧', c:'#5FB8B0'},
    {i:'fa-sun',       e:'☀️', c:'#F0B429'},
    {i:'fa-cloud',     e:'☁️', c:'#8FB4D8'},
    {i:'fa-apple-whole',e:'🍏', c:'#E2665A'},
    {i:'fa-carrot',     e:'🥕', c:'#E2874A'},
    {i:'fa-wheat-awn',  e:'🌾', c:'#C8A24A'},
    {i:'fa-star',       e:'⭐', c:'#E8B33C'},
    {i:'fa-fish',       e:'🐟', c:'#4FA3C7'},
    {i:'fa-feather',    e:'🖋️', c:'#A98BC9'},
    {i:'fa-clover',     e:'🍀', c:'#58A86B'},
    {i:'fa-moon',       e:'🌙', c:'#7B7FC4'},
    {i:'fa-mug-hot',    e:'☕', c:'#A0785A'}
  ];

const GRID=14;

const BASE_SLOT=7;

let SLOT_N=7;

const COMBO_WINDOW=3000;

function mkTile(id,layer,x,y,q,qp){
  return {id,type:0,layer,x,y,q,qp,
    removed:false,inTray:false,matching:false,el:null,biq:false,
    sp:null,frozen:false,chainN:0,bombK:0,bombLeft:0,shell:false,shown:false};
}

function genLevel(lv){
  const cfg=levelDef(lv).cfg;
  const tiles=[];
  /* 1) 每层可用格位（层与层之间半格错位，形成堆叠感） */
  SLOT_N=cfg.slots||BASE_SLOT;
  const shape=cfg.shape||SHAPES[0];
  /* 层间收缩固定为 k：覆盖率是形状影响难度的真正通道，动它会让已定标的模型失效。
    形状裁掉的容量通过「块数上限留出余量」来吸收，而不是靠放宽收缩 */
  const layerSlots=[];
  for(let k=0;k<cfg.layers;k++){
    const lo=k, hi=GRID-2-k;
    if(hi<lo) break;
    const span=Math.max(1,hi-lo);
    const all=[], keep=[];
    for(let x=lo;x<=hi;x+=2) for(let y=lo;y<=hi;y+=2){
      const p={x,y};
      all.push(p);
      /* 归一化到本层区域的 [0,1]²，用同一个遮罩裁剪，整牌轮廓一致 */
      if(shape.m((x-lo)/span,(y-lo)/span)) keep.push(p);
    }
    layerSlots.push(shuffle(keep.length>=Math.min(3,all.length)?keep:all));
  }
  const K=layerSlots.length;
  const caps=layerSlots.map(s=>s.length);
  const totalCap=caps.reduce((a,b)=>a+b,0);
  let N=Math.min(cfg.n,totalCap - totalCap%3);

  /* 2) 分配每层块数：底层最宽、越往上越少且越密，形成金字塔 */

```

```

const w=caps.map((c,k)=>c*(1+0.10*k));
const sw=w.reduce((a,b)=>a+b,0);
const counts=caps.map((c,k)=>Math.min(c,Math.max(1,Math.round(N*w[k]/sw))));
let diff=N-counts.reduce((a,b)=>a+b,0);
for(let k=0;diff>0&&k<K;k++){ const add=Math.min(diff,caps[k]-counts[k]); counts[k]+=add; diff
-=add; }
for(let k=0;diff<0&&k<K;k++){ const sub=Math.min(-diff,counts[k]-1); counts[k]-=sub; diff+=sub
; }

/* 3) 落位 */
let uid=0;
for(let k=0;k<K;k++){
    for(let i=0;i<counts[k];i++){
        const s=layerSlots[k][i];
        tiles.push(mkTile(uid++,k,s.x,s.y,-1,0));
    }
}
/* 4) 底部牌堆 */
const queues=[];
cfg.q.forEach((len,qi)=>{
    const arr=[];
    for(let p=0;p<len;p++){
        const t=mkTile(uid++, -1,0,0,qi,p);
        t.biq=!!cfg.biq;
        arr.push(t); tiles.push(t);
    }
    queues.push(arr);
});

/* 5) 修剪为 3 的倍数（从最上层拿掉多余的） */
while(tiles.length%3!==0){
    let idx=-1,best=-2;
    for(let i=0;i<tiles.length;i++){ if(tiles[i].q<0&&tiles[i].layer>best){best=tiles[i].layer;i
dx=i;}} }
    if(idx<0) break;
    tiles.splice(idx,1);
}

/* 6) 反向求解：先算一条合法的“可消除顺序”，再沿这条顺序染色 — 保证有解 */
const order=solveOrder(tiles,queues);
paintOrder(order,planColors(cfg.types,order.length/3),{},cfg.d||3);
/* 7) 特殊牌：沿着同一条顺序反推能放在哪，保证不会卡死必胜路线 */
applySpecials(order,cfg);
/* 8) 把必胜顺序交出去：关卡目标要用它算配额，失败回放要用它演示 */
return {tiles,queues,order,stats:canonicalStats(order)};
}

const SPECIALS={
    freeze :{icon:'fa-snowflake',    emo:'❄️', name:'冰冻牌', tip:'被冰封住了，先消掉它同层紧挨着的
一张牌就能解冻'},
    chain  :{icon:'fa-link',          emo:'🔗', name:'锁链牌', tip:'锁着的牌，全场再消够指定组数就会
自动解开'},
    dbl    :{icon:'fa-layer-group',   emo:'📦', name:'双层牌', tip:'外面裹了一层草垛，要点两下：第一
下拆壳，第二下才进卡槽'},
    rain   :{icon:'fa-rainbow',       emo:'🌈', name:'彩虹牌', tip:'不占卡槽格子！单独放在上面的彩虹
槽里，凑齐 3 张自动消除'},
    bomb   :{icon:'fa-bomb',          emo:'💣', name:'炸弹牌', tip:'进了卡槽就开始倒数，数完还没消掉
就直接失败'},
    mystery:{icon:'fa-question',      emo:'❓', name:'谜之牌', tip:'花色是盖着的，只有进了卡槽才翻开
，没法提前规划'},

```

```

/* 以下四种只在「被消除时」给奖励，完全不参与规则判定，对有解性零风险 */
coin  :{icon:'fa-coins',      emo:'👉', name:'金币牌', tip:'消掉它会额外掉金币，看到就先拿'},
wool  :{icon:'fa-feather',    emo:'👉', name:'羊毛牌', tip:'消掉它会额外掉羊毛，牧场建设就靠它'},
gift   :{icon:'fa-gift',      emo:'👉', name:'礼盒牌', tip:'消掉它会随机送一个道具'},
torch  :{icon:'fa-lightbulb',  emo:'💡', name:'灯笼牌', tip:'消掉它会照亮 3 张谜之牌，把盖着的花色翻开'},
supply :{icon:'fa-truck-ramp-box', emo:'🚚', name:'补给牌', tip:'牌堆里盖着的补给，全场消够指定组数才会送上来'}
};

const REWARD_SP={coin:1,wool:1,gift:1,torch:1};

function canonicalStats(order){
  const tray=[], leave=new Map(), before=new Array(order.length), trios=[];
  let m=0;
  for(let i=0;i<order.length;i++){
    before[i]=m;
    const t=order[i]; tray.push(t);
    const same=tray.filter(x=>x.type===t.type);
    if(same.length>=3){
      const trio=same.slice(0,3);
      trio.forEach(x=>{ leave.set(x,i); tray.splice(tray.indexOf(x),1); });
      trios.push(trio); m++;
    }
  }
  return {leave,before,triots,groups:m};
}

function neighbors(t,tiles){
  return tiles.filter(o=>o!==t && o.q<0 && o.layer===t.layer &&
    ((Math.abs(o.x-t.x)===2 && o.y===t.y) || (Math.abs(o.y-t.y)===2 && o.x===t.x)));
}

function applySpecials(order,cfg){
  const mix=cfg.sp; if(!mix) return;
  const {leave,before,triots}=canonicalStats(order);
  const pos=new Map(); order.forEach((t,i)=>pos.set(t,i));
  const total=order.length;

  const take=(key,ratio,pick)=>{
    if(!ratio) return;
    let want=Math.round(total*ratio);
    const cand=shuffle(order.filter(t=>!t.sp&&pick(t)));
    for(const t of cand){ if(want<=0) break; t.sp=key; want--; }
  };

  /* 彩虹牌：必须整组三张一起转换，否则原花色会少一张、永远配不齐。
   转换整组之后每种花色的张数仍是 3 的倍数，必胜顺序原样成立 */
  if(mix.rain){
    const want=Math.max(1,Math.round(total*mix.rain/3));
    shuffle(triots.filter(g=>g.every(t=>!t.sp))).slice(0,want)
      .forEach(g=>g.forEach(t=>{ t.sp='rain'; }));
  }

  /* 冰冻：必胜顺序里，它之前必须已经消掉过一个同层邻居，否则永远解不开 */
  take('freeze',mix.freeze,t=>{
    if(t.q>=0) return false;
    const i=pos.get(t);
    return neighbors(t,order).some(o=>pos.get(o)<i);
  }

```

```

    });
    /* 锁链: 锁的组数不能超过它在必胜顺序里之前已完成的组数 */
    take('chain',mix.chain,t=>before[pos.get(t)]>=2);
    order.forEach(t=>{ if(t.sp==='chain'){
        const b=before[pos.get(t)];
        t.chainN=1+rnd(Math.max(1,Math.min(b,4)));
    }});
    /* 炸弹: 倒数必须够它在必胜顺序里撑到自己那组消掉 */
    take('bomb',mix.bomb,t=>{
        const i=pos.get(t), lv=leave.get(t);
        return lv!==undefined && lv-i<=6;
    });
    order.forEach(t=>{ if(t.sp==='bomb'){
        t.bombK=(leave.get(t)-pos.get(t))+3+rnd(3);
    }});
    /* 双层: 只多花一次点击, 不影响顺序 */
    take('dbl',mix.dbl,()=>true);
    order.forEach(t=>{ if(t.sp==='dbl') t.shell=true; });
    /* 谜之: 只藏信息 */
    take('mystery',mix.mystery,()=>true);
    /* 补给牌: 只放在牌堆后半段, 解锁组数由必胜顺序倒推, 保证送上来时还来得及取 */
    take('supply',mix.supply,t=>t.q>=0 && before[pos.get(t)]>=2);
    order.forEach(t=>{ if(t.sp==='supply'){
        const b=before[pos.get(t)];
        t.chainN=1+rnd(Math.max(1,Math.min(b,5)));
    }});
    /* 奖励类: 纯掉落, 不参与任何判定 */
    take('coin',mix.coin,()=>true);
    take('wool',mix.wool,()=>true);
    take('gift',mix.gift,()=>true);
    take('torch',mix.torch,t=>t.q<0);
    /* 迷雾层: 整层盖住, 用的是同一套隐藏逻辑 */
    if(mix.fogLayer!==null){
        order.forEach(t=>{ if(t.q<0&&t.layer===mix.fogLayer&&!t.sp) t.sp='mystery'; });
    }
    order.forEach(t=>{ if(t.sp==='freeze') t.frozen=true; });
}

function planColors(types,m){
    const full=Math.max(1,Math.floor(types)), frac=types-full;
    const w=[]; for(let i=0;i<full;i++) w.push(1);
    if(frac>0.01) w.push(frac);
    const k=Math.min(w.length,m,TYPE_DEFS.length);
    w.length=k;
    const sw=w.reduce((a,b)=>a+b,0);
    const cnt=w.map(x=>Math.max(1,Math.round(m*x/sw)));
    let diff=m-cnt.reduce((a,b)=>a+b,0), guard=999;
    for(let i=0;diff>0&&guard-->0;i=(i+1)%k){ cnt[i]++; diff--; }
    guard=999;
    for(let i=k-1;diff<0&&guard-->0;i=(i+k-1)%k){ if(cnt[i]>1){ cnt[i]--; diff++; } }
    const palette=shuffle(TYPE_DEFS.map((_,x)=>x)).slice(0,k);
    const remain={};
    palette.forEach((c,j)=>remain[c]=cnt[j]*3);
    return remain;
}

function paintOrder(order,remain,tray0,cap){
    const tray=Object.assign({},tray0);
    let size=0; for(const c in tray) size+=tray[c];
    cap=Math.max(3,Math.min(SLOT_N-1,cap),size);

```

```

const at=c=>tray[c]||0;
const one=a=>a[rnd(a.length)];
for(let i=0;i<order.length;i++){
  /* 彩虹牌靠 sp 配对、走独立槽，既不占卡槽也不参与花色配额，直接跳过 */
  if(order[i].sp==='rain') continue;
  const avail=Object.keys(remain).map(Number).filter(c=>remain[c]>0);
  if(!avail.length) break;
  const done =avail.filter(c=>at(c)===2); /* 放下即三连，槽位 -2 */
  const half =avail.filter(c=>at(c)===1); /* 放下变成两张，槽位 +1 */
  const fresh=avail.filter(c=>at(c)===0); /* 开一个新花色，槽位 +1 */
  let c;
  if(size>=cap){
    /* 到顶了必须收缩。下面的 cap-1 规则保证此时 done 一定非空 */
    c = done.length?one(done) : half.length?one(half) : one(fresh);
  }else if(size===cap-1){
    /* 下一步就会顶满，先把某个花色凑到两张，确保届时消得掉 */
    c = half.length?one(half) : fresh.length?one(fresh) : one(done);
  }else{
    /* 还有余量：多开新花色制造压力，偶尔推进已有的 */
    c = (fresh.length && RNG()<.75) ? one(fresh)
      : half.length?one(half) : fresh.length?one(fresh) : one(done);
  }
  order[i].type=c;
  remain[c]--;
  if(at(c)===2){ tray[c]=0; size-=2; } else { tray[c]=at(c)+1; size++; }
}
}

function overlap(a,b){ return Math.abs(a.x-b.x)<2 && Math.abs(a.y-b.y)<2; }

function freeIn(t,set){
  if(t.q>=0){
    /* 双向牌堆：只要是某一端就能取，是约束放宽，有解性天然保持 */
    let smaller=false, larger=false;
    for(const o of set){
      if(o===t||o.q!==t.q) continue;
      if(o.qp<t.qp) smaller=true; else larger=true;
      if(smaller&&(!t.biq||larger)) break;
    }
    return t.biq ? !(smaller&&larger) : !smaller;
  }
  for(const o of set){ if(o!==t && o.q<0 && o.layer>t.layer && overlap(o,t)) return false; }
  return true;
}

function solveOrder(tiles){
  const set=new Set(tiles);
  const order=[];
  let guard=tiles.length+5;
  while(set.size && guard-->0){
    const free=[];
    for(const t of set) if(freeIn(t,set)) free.push(t);
    if(!free.length){ for(const t of set) order.push(t); break; }
    /* 偏向先取高层，视觉上三连更自然 */
    free.sort((a,b)=>(b.layer-a.layer)||((RNG()-.5)));
    const take=free[rnd(Math.max(1,Math.min(free.length,4)))];
    order.push(take); set.delete(take);
  }
  return order;
}

```

```
const WEEKLY=[
  {id:'narrow', n:'窄槽周', d:'卡槽只有 5 格', apply:c=>{ c.slots=5; c.d=Math.min(c.d,4)
; }},
  {id:'colors', n:'缤纷周', d:'花色多一种', apply:c=>{ c.types+=(c.types+1).toFixed(
2); }},
  {id:'fog', n:'迷雾周', d:'一半的牌盖着花色', apply:c=>{ c.sp=Object.assign({},c.sp,{m
ystery:.42}); }},
  {id:'wide', n:'宽槽周', d:'卡槽多两格, 但花色翻倍', apply:c=>{ c.slots=9; c.types+=(c.type
s*1.35).toFixed(2); }},
  {id:'ice', n:'霜冻周', d:'冰冻牌大量出现', apply:c=>{ c.sp=Object.assign({},c.sp,{f
reeze:.26}); }},
  {id:'boom', n:'爆破周', d:'炸弹牌大量出现', apply:c=>{ c.sp=Object.assign({},c.sp,{b
omb:.10}); }}
];

function weekIndex(d){
  d=d||new Date();
  return Math.floor((Date.UTC(d.getFullYear(),d.getMonth(),d.getDate())/86400000+4)/7);
}

function weeklyMod(){ return WEEKLY[weekIndex()%WEEKLY.length]; }

const WEEKLY_LV=18;

function ckChar(s){ let h=7; for(let i=0;i<s.length;i++) h=(h*31+s.charCodeAt(i))%36; return h.t
oString(36).toUpperCase(); }

function makeCode(lv,seed,res){
  let body=(Math.max(1,Math.min(MAX_LEVEL,lv))*16777216 + ((seed>>>0)%16777216)).toString(36).to
UpperCase();
  if(res){
    /* 10ms 精度。原来按整秒四舍五入, 最大 ±500ms 误差,
    势均力敌时慢 400ms 也会被判胜 — 而那正是对战最需要准的区间 */
    const cs=Math.max(0,Math.min(599999,Math.round(res.time/10)));
    const cb=Math.max(0,Math.min(63,res.combo|0));
    body+='+'+(cs*64+cb).toString(36).toUpperCase();
  }
  return body+ckChar(body);
}

function parseCode(c){
  /* 分隔符统一成 '.' 再算校验, 这样 . - _ / 四种写法都认 */
  const raw=String(c).toUpperCase().replace(/\s+/g,'').replace(/[\-_]/g,'.');
  if(raw.length<2) return null;
  const body=raw.slice(0,-1);
  if(ckChar(body)!==raw.slice(-1)) return null;
  const bits=body.split('.').filter(Boolean);
  const head=(bits[0]||'').replace(/^[^0-9A-Z]/g,'');
  const n=parseInt(head,36);
  if(!isFinite(n)||n<=0) return null;
  const lv=Math.floor(n/16777216), seed=n%16777216;
  if(lv<1||lv>MAX_LEVEL) return null;
  let target=null;
  if(bits[1]){
    const m=parseInt(bits[1].replace(/^[^0-9A-Z]/g,''),36);
    if(isFinite(m)&&m>0) target={time:Math.floor(m/64)*10, combo:m%64};
  }
  return {lv,seed,target};
}
```

```
function dailySeed(d){ d=d||new Date(); return d.getFullYear()*10000+(d.getMonth()+1)*100+d.getDate(); }

function waveLevel(w){ return 6+w*2; }

const TOOL_KEYS=['shuffle','undo','out'];

function tileNb(a,b){
    return a.q<0&&b.q<0&&a.layer===b.layer&&
        ((Math.abs(a.x-b.x)===2&&a.y===b.y)|| (Math.abs(a.y-b.y)===2&&a.x===b.x));
}

function solveOrderLocked(rest,startMatches){
    const set=new Set(rest), order=[];
    const frozen=new Map();
    rest.forEach(t=>{ if(t.sp==='freeze'&&t.frozen) frozen.set(t,true); });
    let guard=rest.length+5;
    while(set.size&&guard-->0){
        const m=startMatches+Math.floor(order.length/3);
        const free=[];
        for(const t of set){
            if(!freeIn(t,set)) continue;
            if(t.sp==='freeze'&&frozen.get(t)) continue;
            if((t.sp==='chain'||t.sp==='supply')&&m<t.chainN) continue;
            free.push(t);
        }
        if(!free.length) return null;
        free.sort((a,b)=>(b.layer-a.layer)|| (RNG()-.5));
        const t=free[rnd(Math.max(1,Math.min(free.length,4)))];
        set.delete(t); order.push(t);
        for(const o of set) if(o.sp==='freeze'&&frozen.get(o)&&tileNb(o,t)) frozen.set(o,false);
    }
    return set.size?null:order;
}

function retimeBombs(rest,order,held,rainHeld){
    const tray=held.slice(), rainSlot=rainHeld.slice();
    const pos=new Map(); order.forEach((t,i)=>pos.set(t,i));
    const clearAt=new Map();
    for(let i=0;i<order.length;i++){
        const t=order[i];
        if(t.sp==='rain'){
            rainSlot.push(t);
            if(rainSlot.length>=3){ rainSlot.forEach(x=>clearAt.set(x,i)); rainSlot.length=0; }
            continue;
        }
        tray.push(t);
        const by={}; tray.forEach(x=>{ (by[x.type]=by[x.type]||[]).push(x); });
        let trio=null; for(const k in by) if(by[k].length>=3){ trio=by[k].slice(0,3); break; }
        if(trio) trio.forEach(x=>{ clearAt.set(x,i); tray.splice(tray.indexOf(x),1); });
    }
    let ok=true;
    rest.concat(held).forEach(t=>{
        if(t.sp!=='bomb') return;
        const c=clearAt.get(t);
        if(c===undefined){ ok=false; return; }
        const start=pos.has(t)?pos.get(t):-1;
        let ins=0; /* 倒数按「之后有几张牌入槽」走，彩虹牌不入槽 */
        for(let i=start+1;i<=c;i++) if(order[i]&&order[i].sp!=='rain') ins++;
    });
}
```

```

    t.bombK=ins+2;
    if(start<0) t.bombLeft=t.bombK;          /* 已在卡槽里的炸弹，倒数一并重置 */
  });
  return ok;
}

function verifyPlayable(rest,order,held,rainHeld){
  const set=new Set(rest), tray=held.slice(), rainSlot=rainHeld.slice();
  const frozen=new Map(), bomb=new Map();
  rest.forEach(t=>{ if(t.sp==='freeze'&&t.frozen) frozen.set(t,true); });
  tray.forEach(t=>{ if(t.sp==='bomb') bomb.set(t,t.bombLeft||t.bombK); });
  let m=G.matches;
  for(const t of order){
    if(!freeIn(t,set)) return false;
    if(t.sp==='freeze'&&frozen.get(t)) return false;
    if((t.sp==='chain'||t.sp==='supply')&&m<t.chainN) return false;
    set.delete(t);
    for(const o of set) if(o.sp==='freeze'&&frozen.get(o)&&tileNb(o,t)) frozen.set(o,false);
    if(t.sp==='rain'){ rainSlot.push(t); if(rainSlot.length>=3){ rainSlot.length=0; m++; } continue; }
    tray.push(t);
    if(t.sp==='bomb') bomb.set(t,t.bombK);
    let blew=false;
    bomb.forEach((v,k)=>{ if(k!==t&&tray.includes(k)){ bomb.set(k,v-1); if(v-1<=0) blew=true; } });
  });
  const by={}; tray.forEach(x=>{ (by[x.type]=by[x.type]||[]).push(x); });
  let trio=null; for(const k in by) if(by[k].length>=3){ trio=by[k].slice(0,3); break; }
  if(trio){ trio.forEach(x=>{ tray.splice(tray.indexOf(x),1); bomb.delete(x); }); m++; }
  else if(blew) return false;
  if(tray.length>=SLOT_N) return false;
}
return set.size===0 && tray.length===0 && rainSlot.length===0;
}

function reshuffleSolvable(){
  const rest=G.tiles.filter(t=>!t.removed);
  if(rest.length<3) return 'empty';
  /* 用本局真正的配置。每日/周常/无尽的 G.lv 都是 1，
     直接 levelDef(G.lv) 会回落到第 1 关，把题面洗成幼儿园难度 */
  const cfg=G.rcfg||{types:levelDef(G.lv).cfg.types,d:levelDef(G.lv).cfg.d};
  const cap=Math.max(3,Math.min(SLOT_N-1,(cfg.d||3)-(toolLv('shuffle')-1)));
  const held=G.tray.filter(x=>!x.matching);
  const rainHeld=G.rain.filter(x=>!x.matching);
  const heldCount={};
  held.forEach(t=>heldCount[t.type]=(heldCount[t.type]||0)+1);

  const plain=rest.filter(t=>t.sp!=='rain');    /* 彩虹牌不参与花色配额 */
  const backup=rest.map(t=>t.type);
  const bombBackup=rest.concat(held).map(t=>[t,t.bombK,t.bombLeft]);

  let needTotal=0;
  for(const c in heldCount) needTotal+=3-heldCount[c];
  const triples=(plain.length-needTotal)/3;
  if(triples<0||triples%1!==0) return 'fail';

  for(let attempt=0;attempt<14;attempt++){
    const order=solveOrderLocked(rest,G.matches);
    if(!order) continue;
    const remain={};
    for(const c in heldCount) remain[c]=3-heldCount[c];
  }

```

```

const pool=planColors(cfg.types,triples);
for(const c in pool) remain[c]=(remain[c]||0)+(pool[c]);
/* collect 目标: 保证目标花色还留在盘面上, 否则任务被无声废掉。
   搬运量必须是 3 的倍数, 否则会破坏「每色总数是 3 的倍数」这条不变量 */
const g=G.goal;
if(g&&g.t==='collect'){
  const want=3*Math.max(1,g.need-g.got);
  if((remain[g.type]||0)<want){
    const donor=Object.keys(remain).map(Number)
      .filter(c=>c!==g.type).sort((a,b)=>remain[b]-remain[a])[0];
    if(donor!==null){
      let move=Math.min(want-(remain[g.type]||0),Math.max(0,(remain[donor]||0)-3));
      move=Math.floor(move/3)*3;
      if(move>0){ remain[donor]-=move; remain[g.type]=(remain[g.type]||0)+move; }
    }
  }
}
paintOrder(order,remain,heldCount,cap);
if(!retimeBombs(rest,order,held,rainHeld)) continue;
if(verifyPlayable(rest,order,held,rainHeld)) return 'ok';
}
rest.forEach((t,i)=>{ t.type=backup[i]; }); /* 洗不出可解局面, 原样还回去 */
bombBackup.forEach(([t,k,l])=>{ t.bombK=k; t.bombLeft=l; });
return 'fail';
}

const SAYINGS=[
  '风把云吹散了, 剩下的时间, 都用来陪你喜欢的羊吧。',
  '慢一点也没关系, 草原上的路从来不催人。',
  '今天的第一缕阳光, 已经替你晒暖了牧场。',
  '把烦恼交给风, 把青草留给羊。',
  '走得远不如走得稳, 羊群会一直等你。'
];

const CK_COIN=[20,25,30,40,50,60,100];

const PRICE={shuffle:120,undo:80,out:200};

const HELP_BASE=[
  ['fa-layer-group','怎么算赢','点击盘面上没被压住的牌, 它会飞进下方卡槽; 卡槽里凑齐 3 张同花色自动消除。把盘面、卡槽、彩虹槽全部清空就算过关。'],
  ['fa-table-columns','卡槽只有 7 格','卡槽塞满且没有任何三连时判负。这是唯一的失败方式(限时与限步关卡另算)。周常变体会把格子改成 5 格或 9 格。'],
  ['fa-box-open','底部牌堆','盘面下方的牌堆只能取最前面一张; 少数关卡的牌堆是双向的, 两端都能取。'],
  ['fa-clover','必定有解','每一局都是先算出一条合法的通关顺序, 再沿着它铺花色和特殊牌 — 有解是构造出来的, 不是碰运气。卡在死局只会是取牌顺序的问题, 失败页可以直接看参考解法回放。']
];

const HELP_MODES=[
  /* 章节数按 CHAPTERS.length 说: 50 关 ÷ CH_SIZE(10) = 5 章, 原来写的「四个章节」
   与 CHAPTERS 里实际的五个章节对不上。宝箱关 isBonus 排除了前 12 关, 第一章没有 */
  ['fa-map','闯关','50 关, 五个章节。每章第 10 关是 BOSS 关; 第二章起每章第 5 关是宝箱关, 金币翻倍、必掉图鉴。'],
  ['fa-calendar-day','每日挑战','全服同一套题面, 每天换一次。因为同题, 成绩才有可比性。'],
  ['fa-calendar-week','周常挑战','每周换一套变体规则(窄槽、迷雾、爆破等), 同一周内全服相同。'],
  ['fa-infinity','无尽草原','一波清完接下一波, 每波更难, 中途可选增益。比的是累计消除组数。']
];

```

```
const SHEEP=[
  {id:'hat', n:'草帽羊', r:1, p:{t:'coin',v:3}, c:['#2E6B3E','#FFFFFF','#FBFDF7','#F6C445','#D3E8C6'], s:'第一只跟着你出门的羊,草帽是它自己叼来的。'},
  {id:'moss', n:'苔藓羊', r:1, p:{t:'wool',v:3}, c:['#3D6B4A','#EFF6EC','#F8FCF6','#A8C98F','#CFE3C6'], s:'喜欢在背阴的石头上打盹,毛里总有青苔味。'},
  {id:'cloud', n:'云朵羊', r:1, p:{t:'coin',v:3}, c:['#5A7A8C','#FBFDF7','#FFFFFF','#BBD5E4','#D8E7F0'], s:'走路很轻,远看像一朵挪动的云。'},
  {id:'wheat', n:'麦穗羊', r:1, p:{t:'wool',v:3}, c:['#8A6B2A','#FFF9E8','#FFFD4','#E8C878','#EDE0BE'], s:'秋天最忙,据说能背回半亩地的麦穗。'},
  {id:'mint', n:'薄荷羊', r:1, p:{t:'coin',v:4}, c:['#3E8574','#F0FBF7','#F8FEFC','#9FD9C6','#CEEDE3'], s:'路过的地方都会留下一股清凉气。'},
  {id:'clay', n:'陶土羊', r:1, p:{t:'wool',v:4}, c:['#8A5A3E','#FFF3EA','#FFFAF5','#DBA07A','#EBD3C2'], s:'在河滩长大,蹄子上永远沾着泥。'},
  {id:'dew', n:'露珠羊', r:1, p:{t:'coin',v:4}, c:['#3C7E93','#F0AFD','#F9FDFF','#9AD2E4','#CDE9F2'], s:'天没亮就出门,只为舔到最新鲜的那滴露水。'},
  {id:'straw', n:'干草羊', r:1, p:{t:'wool',v:4}, c:['#9A7B36','#FFFBEE','#FFFD7','#E0C489','#EEE2C4'], s:'冬天的粮仓管理员,睡觉也抱着草垛。'},
  {id:'fern', n:'蕨叶羊', r:1, p:{t:'coin',v:4}, c:['#2F6B4E','#EEF8F1','#F7FCF9','#8CC9A4','#C9E6D5'], s:'认得林子里每一株蕨,从不迷路。'},
  {id:'stone', n:'碎石羊', r:1, p:{t:'wool',v:4}, c:['#5E6670','#F5F6F8','#FBFBFC','#B6BEC8','#DCE0E6'], s:'沉默寡言,撞过的石头比走过的路多。'},
  {id:'peach', n:'蜜桃羊', r:1, p:{t:'coin',v:5}, c:['#A85A5A','#FFF2F2','#FFF9F9','#F0A9A9','#F3D6D6'], s:'夏天会偷偷去果园,回来时脸是红的。'},
  {id:'lilac', n:'紫丁香', r:1, p:{t:'wool',v:5}, c:['#6E5A96','#F7F2FD','#FCFAFF','#C0A8E0','#DED2F0'], s:'黄昏时最好看,毛尖泛着淡淡的紫。'},
  {id:'olive', n:'橄榄羊', r:1, p:{t:'coin',v:5}, c:['#6B7B36','#F7FAEC','#FCFDF6','#BCCB84','#DDE6C2'], s:'不挑食,据说吃过牧场里所有能吃的东西。'},
  {id:'coral', n:'珊瑚羊', r:1, p:{t:'wool',v:5}, c:['#B06248','#FFF4F0','#FFFAF8','#F0AE94','#F5D9CE'], s:'从很远的海边走来,还记得潮水的声音。'},

  {id:'scarf', n:'围巾羊', r:2, p:{t:'combo',v:600}, c:['#2F6C8F','#F4FAFE','#FBFDF7','#EE9A87','#CDE4F2'], s:'围巾是牧羊人织的,它一年四季都不肯摘。'},
  {id:'snow', n:'初雪羊', r:2, p:{t:'combo',v:600}, c:['#5C7A96','#FFFFFF','#FBFDF7','#B9D4E8','#DCEAF4'], s:'落雪那天出生,从此不怕冷。'},
  {id:'amber', n:'琥珀羊', r:2, p:{t:'coin',v:7}, c:['#9A6516','#FFF6E4','#FFFCF2','#E8B34C','#F0DDB0'], s:'毛色像凝固的阳光,靠近会觉得暖。'},
  {id:'ivy', n:'常春羊', r:2, p:{t:'wool',v:7}, c:['#2C6A3C','#EDF7EE','#F7FCF8','#82C08E','#C4E2CA'], s:'爬过最高的墙,也蹭掉过最多的毛。'},
  {id:'thaw', n:'暖阳羊', r:2, p:{t:'thaw',v:2}, c:['#B07A1E','#FFF8E6','#FFFD4','#F2C55C','#F2E3B8'], s:'走到哪儿,哪儿的冰就化了。'},
  {id:'peek', n:'千里羊', r:2, p:{t:'peek',v:2}, c:['#456B9A','#F2F7FE','#FAFCFF','#A8C3E8','#D3E1F5'], s:'站在坡顶能看清三里外的草,眼神好得离谱。'},
  {id:'lucky', n:'四叶羊', r:2, p:{t:'drop',v:30}, c:['#3A8A50','#EFFAF1','#F8FDF9','#8FD3A2','#C8EAD2'], s:'总能踩到四叶草,连它自己都觉得很奇怪。'},

  {id:'star', n:'星月羊', r:3, p:{t:'luck',v:1}, c:['#4B3A78','#F5F1FD','#FCFAFF','#C9A6F0','#DED4F2'], s:'夜里会抬头很久,像在等谁。'},
  {id:'aurora', n:'极光羊', r:3, p:{t:'coin',v:10}, c:['#2F7A86','#EEFBFB','#F8FEFE','#7ED8D8','#C2ECEC'], s:'极夜里见过天上的光,回来后毛就变了颜色。'},
  {id:'golden', n:'鎏金羊', r:3, p:{t:'wool',v:10}, c:['#8A6A12','#FFFAE6','#FFFD3','#F2CE55','#F0E2B4'], s:'传说中被阳光亲吻过的羊,很少有人真的见过。'}
];
```

```
const RARITY=['','普通','稀有','传说'];
```

```
const PASSIVE_TXT={
  coin:v=>'金币产出 '+v+'%',
  wool:v=>'羊毛产出 '+v+'%',
  combo:v=>'连击窗口 '+((v/1000).toFixed(1)+' 秒'),
  thaw:v=>'开局自动解冻 '+v+' 张冰冻牌',
```

```

    peek:v=>'开局自动翻开 '+v+' 张谜之牌',
    drop:v=>'图鉴掉落率 '+v+'%',
    luck:v=>'通关必定掉落一只羊'
  };

const SHARD_NEED=[0,3,8];

const SHARD_MAX=8;

const PITY_N=12;

const EXCHANGE={1:6, 2:12, 3:25};

const FARM=[
  {id:'grass', n:'青草地', ico:'fa-seedling', max:5, cost:l=>60+l*40, desc:l=>'金币产出 '++(1*2)+'%'},
  {id:'mill', n:'风车', ico:'fa-fan', max:5, cost:l=>80+l*50, desc:l=>'羊毛产出 '++(1*2)+'%'},
  {id:'fence', n:'围栏', ico:'fa-border-all',max:5, cost:l=>70+l*45, desc:l=>'商店道具便宜 '++(1*3)+'%'},
  {id:'pen', n:'羊圈', ico:'fa-house-chimney', max:5, cost:l=>90+l*60, desc:l=>'图鉴掉落率 '++(1*4)+'%'}
];

const TOOL_UP={
  shuffle:{n:'洗牌',ico:'fa-shuffle', cost:[0,120,260], desc:['有解洗牌','有解洗牌 · 局面更宽松','有解洗牌 · 洗成最宽松局面']},
  undo: {n:'撤回',ico:'fa-rotate-left',cost:[0,100,220], desc:['撤回 1 步','连续撤回 2 步','连续撤回 3 步']},
  out: {n:'移出',ico:'fa-arrow-right-from-bracket',cost:[0,140,300], desc:['移出 3 张','移出 4 张','移出 5 张']}
};

const ACH=[
  ['first','初次放羊','通关第 1 关',s=>s.cleared>=1,20],
  ['c5','小有成就','累计通关 5 关',s=>s.cleared>=5,20],
  ['c15','熟能生巧','累计通关 15 关',s=>s.cleared>=15,30],
  ['c40','牧场老手','累计通关 40 关',s=>s.cleared>=40,50],
  ['c80','草原传说','累计通关 80 关',s=>s.cleared>=80,80],
  ['lv5','越走越远','解锁到第 5 关',s=>s.level>=5,20],
  ['lv12','出了新手村','解锁到第 12 关',s=>s.level>=12,30],
  ['lv21','翻过雪线','解锁到第 21 关',s=>s.level>=21,50],
  ['lv31','走到星夜','解锁到第 31 关',s=>s.level>=31,80],
  ['st10','攒星星','累计 10 颗星',s=>starTotal()>=10,20],
  ['st30','三十而立','累计 30 颗星',s=>starTotal()>=30,40],
  ['st60','星光满地','累计 60 颗星',s=>starTotal()>=60,70],
  ['st99','星河灿烂','累计 99 颗星',s=>starTotal()>=99,120],
  ['perfect','一尘不染','有 1 关拿到三星',s=>Object.values(s.stars|{|}).some(v=>v>=3),25],
  ['perfect5','五星连珠','有 5 关拿到三星',s=>Object.values(s.stars|{|}).filter(v=>v>=3).length>=5,45],
  ['perfect15','完美主义','有 15 关拿到三星',s=>Object.values(s.stars|{|}).filter(v=>v>=3).length>=15,90],
  ['combo5','小连击','单局连击达到 5',s=>(s.st.bestCombo|{|})>=5,20],
  ['combo10','连击好手','单局连击达到 10',s=>(s.st.bestCombo|{|})>=10,35],
  ['combo20','连击大师','单局连击达到 20',s=>(s.st.bestCombo|{|})>=20,70],
  ['m100','消了一百组','累计消除 100 组',s=>(s.st.matches|{|})>=100,20],
  ['m500','消了五百组','累计消除 500 组',s=>(s.st.matches|{|})>=500,40],
  ['m2000','消除机器','累计消除 2000 组',s=>(s.st.matches|{|})>=2000,90],
  ['boss1','初见 Boss','通关 1 个 Boss 关',s=>(s.st.boss|{|})>=1,40],
  ['boss3','Boss 克星','通关 3 个 Boss 关',s=>(s.st.boss|{|})>=3,80],

```

```
[ 'notool', '徒手过关', '不用任何道具通关 1 次', s=>(s.st.noTool||0)>=1,25],
[ 'notool10', '老手风范', '不用道具通关 10 次', s=>(s.st.noTool||0)>=10,60],
[ 'daily1', '今日打卡', '完成 1 次每日挑战', s=>(s.st.daily||0)>=1,25],
[ 'daily7', '坚持一周', '完成 7 次每日挑战', s=>(s.st.daily||0)>=7,70],
[ 'endless20', '无尽入门', '无尽模式消 20 组', s=>(s.endless||0)>=20,25],
[ 'endless60', '无尽好手', '无尽模式消 60 组', s=>(s.endless||0)>=60,60],
[ 'dex5', '小小收藏家', '收集 5 只羊', s=>Object.keys(s.dex||{}).length>=5,30],
[ 'dex12', '半个图鉴', '收集 12 只羊', s=>Object.keys(s.dex||{}).length>=12,60],
[ 'dex24', '图鉴全满', '收集全部 24 只羊', s=>Object.keys(s.dex||{}).length>=24,150],
[ 'sr', '传说降临', '收集到 1 只传说羊', s=>Object.keys(s.dex||{}).some(id=>SHEEP_BY[id]&&SHEEP_BY[id].r===3),80],
[ 'farm5', '初具规模', '牧场建设累计 5 级', s=>Object.values(s.farm||{}).reduce((a,b)=>a+b,0)>=5,40],
[ 'farm20', '牧场主', '牧场全部建满', s=>Object.values(s.farm||{}).reduce((a,b)=>a+b,0)>=20,120]
];
```

```
const TITLES=[
  ['', '牧羊新手', ()=>true],
  ['c15', '熟练牧人', null],
  ['c40', '牧场老手', null],
  ['c80', '草原传说', null],
  ['perfect15', '完美主义者', null],
  ['combo20', '连击大师', null],
  ['boss3', 'Boss 克星', null],
  ['daily7', '风雨无阻', null],
  ['dex24', '图鉴收藏家', null],
  ['sr', '传说饲主', null],
  ['farm20', '牧场主', null],
  ['st99', '星河灿烂', null]
];
```

```
const TASKS=[
  {id:'match', stat:'matches', goals:[20,30,45], txt:g=>'累计消除 '+g+' 组', coin:70, wool:14},
  {id:'clear', stat:'clears', goals:[1,2,3], txt:g=>'通关 '+g+' 关', coin:90, wool:18},
  {id:'combo', stat:'combo', goals:[4,5,6], txt:g=>'单局达成 '+g+' 连击', coin:80, wool:16, max:true},
  {id:'play', stat:'plays', goals:[3,5,7], txt:g=>'开局 '+g+' 次', coin:55, wool:11},
  {id:'spec', stat:'spec', goals:[6,10,15], txt:g=>'消掉 '+g+' 张特殊牌', coin:85, wool:17},
  {id:'daily', stat:'dailyRun',goals:[1], txt:()=>>'完成一次每日挑战', coin:110, wool:22}
];
```

```
const BUFFS=[
  {id:'slot', ico:'fa-table-columns', n:'多一格卡槽', d:'接下来每一波都多一个格子，效果会叠加'}
,
  {id:'color', ico:'fa-palette', n:'少一种花色', d:'下一波开始花色少一种，配对更容易'},
  {id:'small', ico:'fa-compress', n:'盘面缩小', d:'下一波少 12 张牌，跑得更快'},
  {id:'tool', ico:'fa-toolbox', n:'补给两件', d:'立刻获得 2 个随机道具'},
  {id:'combo', ico:'fa-fire', n:'连击更宽松', d:'连击窗口延长 1.5 秒，更容易连起来'}
];
```

```
var CORE = {
  G: G,
  SLOT_N: SLOT_N,
  BASE_SLOT: BASE_SLOT,
  GRID: GRID,
```

```

TYPE_DEFS: TYPE_DEFS,
CH_SIZE: CH_SIZE,
CHAPTERS: CHAPTERS,
SHAPES: SHAPES,
GOALS: GOALS,
SPECIALS: SPECIALS,
REWARD_SP: REWARD_SP,
LEVELS: LEVELS,
MAX_LEVEL: MAX_LEVEL,
WEEKLY: WEEKLY,
COMBO_WINDOW: COMBO_WINDOW,
BUFFS: BUFFS,
mulberry32: mulberry32,
seedRNG: seedRNG,
rnd: rnd,
shuffle: shuffle,
chapterIdx: chapterIdx,
chapterOf: chapterOf,
chapterTitle: chapterTitle,
levelName: levelName,
isBoss: isBoss,
isBonus: isBonus,
shapeOf: shapeOf,
rawLevelDef: rawLevelDef,
levelDef: levelDef,
goalOf: goalOf,
makeGoal: makeGoal,
mkTile: mkTile,
overlap: overlap,
freeIn: freeIn,
tileNb: tileNb,
neighbors: neighbors,
planColors: planColors,
spMix: spMix,
waveLevel: waveLevel,
solveOrder: solveOrder,
solveOrderLocked: solveOrderLocked,
canonicalStats: canonicalStats,
genLevel: genLevel,
makeCode: makeCode,
parseCode: parseCode,
weekIndex: weekIndex,
weeklyMod: weeklyMod,
dailySeed: dailySeed,
reshuffleSolvable: reshuffleSolvable,
paintOrder: paintOrder,
applySpecials: applySpecials,
SHEEP: SHEEP,
RARITY: RARITY,
PASSIVE_TXT: PASSIVE_TXT,
SHARD_NEED: SHARD_NEED,
SHARD_MAX: SHARD_MAX,
PITY_N: PITY_N,
EXCHANGE: EXCHANGE,
TOOL_UP: TOOL_UP,
PRICE: PRICE,
TOOL_KEYS: TOOL_KEYS,
FARM: FARM,
ACH: ACH,
TITLES: TITLES,

```

```

    CK_COIN: CK_COIN,
    TASKS: TASKS,
    HELP_BASE: HELP_BASE,
    HELP_MODES: HELP_MODES,
    SAYINGS: SAYINGS
};
if (typeof module !== 'undefined' && module.exports) module.exports = CORE;
else if (typeof globalThis !== 'undefined') globalThis.CORE = CORE;

/* ===== 微信小游戏版 minigame/js/scene_game.js ===== */
/* 牌局场景。
   规则判定（遮挡、牌堆、冰冻解冻、锁链/补给、炸弹倒数、彩虹独立槽、卡槽上限）
   逐条照网页版搬过来，只是把 DOM 换成 canvas 绘制。 */
(function () {
    var CORE = (typeof require !== 'undefined') ? require('./core.js') : globalThis.CORE;
    var UI = (typeof require !== 'undefined') ? require('./ui.js') : globalThis.UI;
    var SAVE = (typeof require !== 'undefined') ? require('./save.js') : globalThis.SAVE;
    var SFX = (typeof require !== 'undefined') ? require('./sfx.js') : globalThis.SFX;
    var C = UI.C;

    var ANIM = 260;           // 入槽飞行时间(ms)
    var CLEAR = 300;         // 三连消除停留(ms)

    function GameScene(lv, seed, mode) {
        this.lv = lv;
        this.seed = seed;
        this.mode = mode || 'normal';
        this.stage = null;
    }

    GameScene.prototype.enter = function (stage) {
        this.stage = stage;
        this.reset();
    };

    GameScene.prototype.reset = function () {
        CORE.G.mode = this.mode;
        CORE.G.wave = this.wave || 1;
        CORE.G.buffs = this.buffs || null;
        CORE.G.lv = this.lv;
        CORE.G.toolLv = SAVE.d.toolLv;
        // 每日/周常用固定种子，全服同题
        if (this.mode === 'daily') this.seed = CORE.dailySeed();
        else if (this.mode === 'weekly') this.seed = CORE.weekIndex() * 7919;
        else if (this.seed == null) this.seed = (Math.random() * 16777216) | 0;
        CORE.seedRNG(this.seed);
        var gen = CORE.genLevel(this.lv);
        this.tiles = gen.tiles;
        this.queues = gen.queues;
        this.order = gen.order;
        this.def = CORE.levelDef(this.lv);
        this.slotN = this.def.cfg.slots || CORE.BASE_SLOT;
        this.tray = [];
        this.rain = [];
        this.matches = 0;
        this.total = this.tiles.length;
        this.over = 0;           // 0 进行中 1 胜 -1 负
        this.combo = 0;
        this.maxCombo = 0;
    }

```

```

    this.lastMatch = 0;
    this.startAt = Date.now();
    this.flying = []; // {t, x0,y0,x1,y1, t0}
    this.clearing = null; // {trio, t0}
    this.history = []; // 撤回用
    this.toolUsed = 0;
    this.goal = gen.goal || (CORE.G.goal = null);
    this.limit = (this.mode === 'daily' || this.mode === 'weekly' || CORE.isBoss(this.lv)) ? 150
: 0;
    // 洗牌要用本局真实配置, 直接查 levelDef 会在每日/周常里回落到第 1 关
    CORE.G.rcfg = { types: this.def.cfg.types, d: this.def.cfg.d };
    this.syncCore();
    this.layout();
};

/* 有解洗牌读的是 CORE.G, 调用前必须同步 */
GameScene.prototype.syncCore = function () {
    CORE.G.tiles = this.tiles;
    CORE.G.queues = this.queues;
    CORE.G.tray = this.tray;
    CORE.G.rain = this.rain;
    CORE.G.matches = this.matches;
    CORE.G.lv = this.lv;
    CORE.G.goal = this.goal;
    CORE.G.toolLv = SAVE.d.toolLv;
};

/* --- 三个道具 --- */
GameScene.prototype.useShuffle = function () {
    if (this.over) return;
    var S = SAVE.d;
    if (!S.tools.shuffle) { this.stage.toast('洗牌用完了, 去商店补一点'); SFX.deny(); return; }
    this.syncCore();
    var r = CORE.reshuffleSolvable();
    if (r !== 'ok') {
        // 洗不出有解排布就如实告诉玩家, 并且不扣道具
        this.stage.toast(r === 'empty' ? '牌太少了, 洗不动' : '这个局面洗不出有解排布, 没有扣道具'
);
        SFX.deny();
        return;
    }
    S.tools.shuffle--; SAVE.save();
    this.toolUsed++;
    this.stage.toast('已重新洗牌 · 保证仍有解');
    SFX.coin();
};

GameScene.prototype.useUndo = function () {
    if (this.over) return;
    var S = SAVE.d;
    if (!S.tools.undo) { this.stage.toast('撤回用完了'); SFX.deny(); return; }
    var n = S.toolLv.undo || 1, done = 0;
    for (var i = 0; i < n; i++) {
        // 只能撤回还留在槽里的牌。history 里也有已经消掉的,
        // 照单全收会把消掉的牌复活回盘面, 那是比撤不动更糟的错
        var t = null;
        while (this.history.length) {
            var cand = this.history[this.history.length - 1];
            this.history.pop();
            if (this.tray.indexOf(cand) >= 0 || this.rain.indexOf(cand) >= 0) { t = cand; break; }
        }
    }
}

```

```

        if (!t) break;
        var idx = this.tray.indexOf(t);
        if (idx < 0) { var ri = this.rain.indexOf(t); if (ri >= 0) this.rain.splice(ri, 1); }
        else this.tray.splice(idx, 1);
        t.removed = false;
        if (t.sp === 'bomb') t.bombLeft = t.bombK;    // 退回盘面，倒数重置
        done++;
    }
    if (!done) { this.stage.toast('还没有可撤回的操作'); SFX.deny(); return; }
    S.tools.undo--; SAVE.save();
    this.toolUsed++;
    this.stage.toast('已撤回 ' + done + ' 步');
};

GameScene.prototype.useOut = function () {
    if (this.over) return;
    var S = SAVE.d;
    if (!S.tools.out) { this.stage.toast('移出用完了'); SFX.deny(); return; }
    var n = 2 + (S.toolLv.out || 1);
    var list = this.tray.filter(function (t) { return true; }).slice(0, n);
    if (!list.length) { this.stage.toast('卡槽是空的'); SFX.deny(); return; }
    var self = this;
    list.forEach(function (t) {
        self.tray.splice(self.tray.indexOf(t), 1);
        t.removed = false;
        if (t.sp === 'bomb') t.bombLeft = t.bombK;
        var h = self.history.indexOf(t); if (h >= 0) self.history.splice(h, 1);
    });
    S.tools.out--; SAVE.save();
    this.toolUsed++;
    this.stage.toast('已移出 ' + list.length + ' 块');
};

/* --- 几何：与网页版 computeT 同一套算法 --- */
GameScene.prototype.layout = function () {
    var P = this.stage.plat;
    var boardTop = 108;
    var boardH = P.H - boardTop - 250;
    var W = P.W - 24;
    var rows = this.queues.length;
    var maxLen = 0;
    for (var i = 0; i < this.queues.length; i++) maxLen = Math.max(maxLen, this.queues[i].length);
};

var T = Math.min(W / 7, (boardH - rows * 6 - 8) / (7 + rows * 1.12));
if (maxLen > 1) T = Math.min(T, W / (1 + 0.30 * (maxLen - 1)));
T = Math.max(26, Math.floor(T));
this.T = T;
this.boardX = (P.W - 7 * T) / 2;
this.boardY = boardTop;
this.qStep = T * 0.30;
this.qY = boardTop + 7 * T + 10;
this.slotY = P.H - 150;
this.rainY = P.H - 200;
};

GameScene.prototype.tileRect = function (t) {
    var T = this.T;
    if (t.q >= 0) {
        var q = this.queues[t.q];
        var w = T + this.qStep * (q.length - 1);
        return { x: (this.stage.plat.W - w) / 2 + t.q * this.qStep, y: this.qY + t.q * (T + 6), w

```

```

: T, h: T }];
    }
    return { x: this.boardX + t.x * T / 2, y: this.boardY + t.y * T / 2, w: T, h: T };
};

/* --- 规则判定（照搬网页版） --- */
GameScene.prototype.isCovered = function (t) {
    if (t.q >= 0) {
        var q = this.queues[t.q], smaller = false, larger = false;
        for (var i = 0; i < q.length; i++) {
            var o = q[i];
            if (o.removed || o === t) continue;
            if (o.qp < t.qp) smaller = true; else larger = true;
        }
        return t.biq ? (smaller && larger) : smaller;
    }
    for (var j = 0; j < this.tiles.length; j++) {
        var b = this.tiles[j];
        if (!b.removed && b.q < 0 && b.layer > t.layer && CORE.overlap(b, t)) return true;
    }
    return false;
};
GameScene.prototype.lockedBy = function (t) {
    if (t.sp === 'freeze' && t.frozen) return 'freeze';
    if (t.sp === 'chain' && this.matches < t.chainN) return 'chain';
    if (t.sp === 'supply' && this.matches < t.chainN) return 'supply';
    return null;
};
GameScene.prototype.thawAround = function (t) {
    if (t.q >= 0) return;
    var self = this;
    this.tiles.forEach(function (o) {
        if (o.removed || o.sp !== 'freeze' || !o.frozen || o.layer !== t.layer) return;
        if ((Math.abs(o.x - t.x) === 2 && o.y === t.y) || (Math.abs(o.y - t.y) === 2 && o.x === t.x)) o.frozen = false;
    });
};

/* --- 点击一张牌 --- */
GameScene.prototype.tap = function (t) {
    if (this.over || this.clearing) return;
    if (t.removed || this.isCovered(t)) return;
    var lk = this.lockedBy(t);
    if (lk) {
        this.stage.toast(lk === 'freeze' ? '这张牌被冰封了，先消掉它旁边的牌'
            : '还没解锁，再多消几组');
        return;
    }
    if (t.sp === 'dbl' && t.shell) { // 双层牌：第一下只拆壳
        t.shell = false;
        return;
    }
    if (t.sp === 'rain') {
        if (this.rain.length >= 3) return;
        t.removed = true;
        this.rain.push(t);
        this.history.push(t);
        SFX.pick();
        this.thawAround(t);
        if (this.rain.length === 3) { this.rain.length = 0; this.registerMatch(); }
    }
};

```

```

        this.checkEnd();
        return;
    }
    if (this.tray.length >= this.slotN) return;

    t.removed = true;
    this.tray.push(t);
    this.history.push(t);
    SFX.pick();
    this.thawAround(t);
    if (t.sp === 'bomb') t.bombLeft = t.bombK;

    var r = this.tileRect(t);
    var slot = this.slotRect(this.tray.length - 1);
    this.flying.push({ t: t, x0: r.x, y0: r.y, x1: slot.x, y1: slot.y, t0: Date.now() });

    // 炸弹倒数: 每有一张牌入槽, 槽里其它炸弹减 1
    for (var i = 0; i < this.tray.length; i++) {
        var x = this.tray[i];
        if (x === t || x.sp !== 'bomb') continue;
        x.bombLeft--;
        if (x.bombLeft <= 0) { this.lose('bomb'); return; }
    }

    var same = this.tray.filter(function (o) { return o.type === t.type; });
    if (same.length >= 3) {
        this.clearing = { trio: same.slice(0, 3), t0: Date.now() };
    } else {
        this.checkEnd();
    }
};

GameScene.prototype.registerMatch = function () {
    var now = Date.now();
    this.combo = (now - this.lastMatch < CORE.COMBO_WINDOW) ? this.combo + 1 : 1;
    this.lastMatch = now;
    this.maxCombo = Math.max(this.maxCombo, this.combo);
    this.matches++;
    CORE.G.matches = this.matches;      // 锁链/补给要看这个数
    SFX.match(this.combo);
};

GameScene.prototype.win = function () {
    if (this.over) return;
    this.over = 1;
    SFX.win();
    var secs = Math.round((Date.now() - this.startAt) / 1000);
    if (this.mode === 'normal') {
        this.reward = SAVE.winLevel(this.lv, secs, this.maxCombo, this.matches, this.toolUsed ===
0);
        this.newAch = SAVE.achDone();
    } else if (this.mode === 'daily' || this.mode === 'weekly') {
        var store = this.mode === 'weekly' ? SAVE.d.weekly : SAVE.d.daily;
        var key = String(this.mode === 'weekly' ? CORE.weekIndex() : CORE.dailySeed());
        var prev = store[key];
        store[key] = {
            time: (!prev || secs * 1000 < prev.time) ? secs * 1000 : prev.time,
            combo: Math.max((prev && prev.combo) || 0, this.maxCombo), done: 1
        };
        SAVE.d.coins += 120;
    }
};

```

```

        SAVE.save();
        this.reward = { coin: 120, wool: 0, stars: 3 };
    }
};
GameScene.prototype.lose = function (why) {
    if (this.over) return;
    this.over = -1;
    this.deathBy = why;
    SFX.fail();
    if (this.mode === 'endless') {
        var total = (this.score || 0) + this.matches;
        if (total > SAVE.d.endless) SAVE.d.endless = total;
        SAVE.d.endlessLog.unshift({ score: total, wave: this.wave || 1, t: '' });
        SAVE.d.endlessLog = SAVE.d.endlessLog.slice(0, 10);
        SAVE.d.coins += total * 2;
        SAVE.save();
    }
};

GameScene.prototype.checkEnd = function () {
    var left = 0;
    for (var i = 0; i < this.tiles.length; i++) if (!this.tiles[i].removed) left++;
    if (!left && !this.tray.length && !this.rain.length) {
        if (this.mode === 'endless') { this.nextWave(); return; }
        this.win();
        return;
    }
    if (this.tray.length >= this.slotN) {
        // 卡槽满且没有任何两张同色 → 死局
        var cnt = {};
        for (var j = 0; j < this.tray.length; j++) cnt[this.tray[j].type] = (cnt[this.tray[j].type] || 0) + 1;
        var can = false;
        for (var k in cnt) if (cnt[k] >= 2) can = true;
        if (!can) this.lose('full');
    }
};

/* 无尽：清完一波接下一波，卡槽此刻必空，串联是安全的 */
GameScene.prototype.nextWave = function () {
    this.score = (this.score || 0) + this.matches;
    this.wave = (this.wave || 1) + 1;
    this.seed = (Math.random() * 16777216) | 0;
    this.stage.toast('第 ' + this.wave + ' 波');
    this.reset();
};

GameScene.prototype.update = function (now) {
    var i;
    if (this.limit && !this.over) {
        var left = this.limit - Math.floor((now - this.startAt) / 1000);
        if (left <= 0) this.lose('time');
    }
    for (i = this.flying.length - 1; i >= 0; i--) {
        if (now - this.flying[i].t0 > ANIM) this.flying.splice(i, 1);
    }
    if (this.clearing && now - this.clearing.t0 > CLEAR) {
        var trio = this.clearing.trio, self = this;
        trio.forEach(function (x) {
            var idx = self.tray.indexOf(x);

```

```

        if (idx >= 0) self.tray.splice(idx, 1);
    });
    this.clearing = null;
    this.registerMatch();
    this.checkEnd();
}
};

/* --- 绘制 --- */
GameScene.prototype.slotRect = function (i) {
    var P = this.stage.plat;
    var n = this.slotN, gap = 4;
    var w = Math.min(40, (P.W - 36 - gap * (n - 1)) / n);
    var total = w * n + gap * (n - 1);
    return { x: (P.W - total) / 2 + i * (w + gap), y: this.slotY, w: w, h: w * 1.12 };
};

GameScene.prototype.drawTile = function (ctx, t, x, y, w, covered) {
    var r = w * 0.18;
    UI.fillRound(ctx, x, y, w, w, r, covered ? '#EEF4EA' : C.paper);
    UI.strokeRound(ctx, x, y, w, w, r, covered ? '#DCE7D2' : C.line, 2);
    var def = CORE.TYPE_DEFS[t.type % CORE.TYPE_DEFS.length];
    ctx.globalAlpha = covered ? 0.35 : 1;
    UI.text(ctx, def.e, x + w / 2, y + w / 2 + 1, { size: w * 0.46, align: 'center' });
    ctx.globalAlpha = 1;
    if (t.sp && t.sp !== 'plain') this.drawBadge(ctx, t, x, y, w, covered);
};

GameScene.prototype.drawBadge = function (ctx, t, x, y, w, covered) {
    var sp = CORE.SPECIALS[t.sp];
    if (!sp) return;
    var s = Math.max(13, w * 0.34);
    var bx = x + w - s * 0.55, by = y - s * 0.2;
    if (t.sp === 'bomb') {
        var left = (t.bombLeft !== null && t.bombLeft > 0) ? t.bombLeft : t.bombK;
        UI.fillRound(ctx, bx - s / 2, by, s, s, s / 2, left <= 3 ? C.danger : C.soil);
        UI.text(ctx, String(left), bx, by + s / 2, { size: s * 0.62, bold: true, color: '#fff', align: 'center' });
        return;
    }
    if (t.sp === 'chain' || t.sp === 'supply') {
        UI.fillRound(ctx, bx - s / 2, by, s, s, s / 2, C.soil);
        UI.text(ctx, String(t.chainN), bx, by + s / 2, { size: s * 0.58, bold: true, color: '#fff', align: 'center' });
        return;
    }
    ctx.globalAlpha = covered ? 0.4 : 1;
    UI.fillRound(ctx, bx - s / 2, by, s, s, s / 2, '#FFFFFF');
    UI.strokeRound(ctx, bx - s / 2, by, s, s, s / 2, C.line, 1.2);
    UI.text(ctx, sp.emo, bx, by + s / 2, { size: s * 0.6, align: 'center' });
    ctx.globalAlpha = 1;
};

GameScene.prototype.draw = function (ctx, stage) {
    var P = stage.plat, self = this;
    var T = this.T;

    // 顶部: 关卡名与进度
    UI.text(ctx, this.def.name || CORE.levelName(this.lv), P.W / 2, 44, { size: 16, bold: true, align: 'center' });

```

```

    if (this.limit) {
        var left = Math.max(0, this.limit - Math.floor((stage.now - this.startAt) / 1000));
        UI.text(ctx, '🕒 ' + Math.floor(left / 60) + ':' + ('0' + (left % 60)).slice(-2), P.W - 16,
44,
            { size: 13, bold: true, color: left <= 20 ? C.danger : C.soil, align: 'right' });
    } else {
        UI.text(ctx, this.mode === 'normal' ? ('第 ' + this.lv + ' 关') : ('第 ' + (this.wave || 1
) + ' 波'),
            P.W - 16, 44, { size: 12, color: C.dim, align: 'right' });
    }
    var done = this.total - this.tiles.filter(function (t) { return !t.removed; }).length;
    var pw = P.W - 32;
    UI.fillRound(ctx, 16, 62, pw, 8, 4, C.mint);
    UI.fillRound(ctx, 16, 62, pw * (done / this.total), 8, 4, C.leaf);
    UI.text(ctx, done + '/' + this.total, P.W - 16, 82, { size: 11, color: C.dim, align: 'right'
});
    if (this.combo > 1) UI.text(ctx, '连击 ×' + this.combo, 16, 82, { size: 11, color: C.soil })
;

// 盘面：按层从下往上画，飞行中的牌单独画
var flying = {};
this.flying.forEach(function (f) { flying[f.t.id] = f; });
var board = this.tiles.filter(function (t) { return t.q < 0; })
    .sort(function (a, b) { return a.layer - b.layer; });
board.forEach(function (t) {
    if (t.removed && !flying[t.id]) return;
    if (flying[t.id]) return;
    var r = self.tileRect(t);
    var cov = self.isCovered(t) || !!self.lockedBy(t);
    self.drawTile(ctx, t, r.x, r.y, T, cov);
    if (!cov && !t.removed) stage.hits.add(r.x, r.y, T, T, function () { self.tap(t); });
});

// 底部牌堆
this.queues.forEach(function (q) {
    q.slice().sort(function (a, b) { return b.qp - a.qp; }).forEach(function (t) {
        if (t.removed && !flying[t.id]) return;
        if (flying[t.id]) return;
        var r = self.tileRect(t);
        var cov = self.isCovered(t) || !!self.lockedBy(t);
        self.drawTile(ctx, t, r.x, r.y, T, cov);
        if (!cov && !t.removed) stage.hits.add(r.x, r.y, T, T, function () { self.tap(t); });
    });
});

// 彩虹槽
var rw = P.W - 32;
UI.fillRound(ctx, 16, this.rainY, rw, 34, 17, '#F3ECFB');
UI.text(ctx, '彩虹槽', 30, this.rainY + 17, { size: 11, bold: true, color: '#7A5AA8' });
for (var i = 0; i < 3; i++) {
    var bx = 80 + i * 32;
    UI.strokeRound(ctx, bx, this.rainY + 5, 24, 24, 6, '#C9B3E6', 1.2);
    if (this.rain[i]) UI.text(ctx, '🌈', bx + 12, this.rainY + 17, { size: 14, align: 'center'
});
}

// 卡槽
for (var s = 0; s < this.slotN; s++) {
    var sr = this.slotRect(s);
    UI.fillRound(ctx, sr.x, sr.y, sr.w, sr.w, sr.w * 0.18, '#F1F6EC');
}

```

```

    UI.strokeRound(ctx, sr.x, sr.y, sr.w, sr.w, sr.w * 0.18, C.line, 1.2);
}
this.tray.forEach(function (t, i) {
    if (flying[t.id]) return;
    var sr = self.slotRect(i);
    var fading = self.clearing && self.clearing.trio.indexOf(t) >= 0;
    ctx.globalAlpha = fading ? 0.45 : 1;
    self.drawTile(ctx, t, sr.x, sr.y, sr.w, false);
    ctx.globalAlpha = 1;
});

// 飞行中的牌
this.flying.forEach(function (f) {
    var k = Math.min(1, (stage.now - f.t0) / ANIM);
    var e = 1 - Math.pow(1 - k, 3);
    var sr = self.slotRect(self.tray.indexOf(f.t));
    var x = f.x0 + (sr.x - f.x0) * e, y = f.y0 + (sr.y - f.y0) * e;
    var w = T + (sr.w - T) * e;
    self.drawTile(ctx, f.t, x, y, w, false);
});

// 三个道具
var S = SAVE.d;
var tools = [['🔀', '洗牌', S.tools.shuffle, function () { self.useShuffle(); }],
    ['↶', '撤回', S.tools.undo, function () { self.useUndo(); }],
    ['👉', '移出', S.tools.out, function () { self.useOut(); }]];
var tw = 64, gap = 18, tx0 = (P.W - tw * 3 - gap * 2) / 2, ty = P.H - 76;
tools.forEach(function (t, i) {
    var x = tx0 + i * (tw + gap);
    UI.fillRound(ctx, x, ty, tw, 52, 15, C.paper);
    UI.strokeRound(ctx, x, ty, tw, 52, 15, C.line, 1.5);
    UI.text(ctx, t[0], x + tw / 2, ty + 19, { size: 19, align: 'center' });
    UI.text(ctx, t[1], x + tw / 2, ty + 41, { size: 10.5, align: 'center', color: C.dim });
    // 剩余数量角标
    UI.fillRound(ctx, x + tw - 16, ty - 7, 22, 18, 9, t[2] > 0 ? C.sun : '#D8DED4');
    UI.text(ctx, String(t[2]), x + tw - 5, ty + 2, { size: 11, bold: true, align: 'center', color: t[2] > 0 ? '#6B4E12' : '#8B958A' });
    stage.hits.add(x, ty, tw, 52, t[3], 4);
});

// 返回
UI.text(ctx, '< 返回', 16, 44, { size: 13, color: C.forest });
stage.hits.add(8, 28, 66, 34, function () { stage.nav(self.mode === 'normal' ? 'levels' : 'home'); });

if (this.over) this.drawResult(ctx, stage);
};

GameScene.prototype.drawResult = function (ctx, stage) {
    var P = stage.plat, self = this;
    ctx.fillStyle = 'rgba(38,60,45,0.45)';
    ctx.fillRect(0, 0, P.W, P.H);
    var w = P.W - 64, h = 292, x = 32, y = (P.H - h) / 2;
    UI.fillRound(ctx, x, y, w, h, 22, C.paper);
    var win = this.over === 1;
    UI.text(ctx, win ? '通关!' : '这一局没过去', P.W / 2, y + 48, { size: 22, bold: true, align: 'center', color: win ? C.forest : C.soil });
    var secs = Math.round((Date.now() - this.startAt) / 1000);
    UI.text(ctx, win ? ('用时 ' + Math.floor(secs / 60) + ':' + ('0' + (secs % 60)).slice(-2) + ' 最高连击 ' + this.maxCombo)

```

```

        : (this.deathBy === 'bomb' ? '炸弹倒数归零了'
        : this.deathBy === 'time' ? '时间到了'
        : '卡槽被占满，没有可消的三张'),
        P.W / 2, y + 88, { size: 13, color: C.dim, align: 'center' });

    if (win && this.reward) {
        var st = '';
        for (var s = 0; s < 3; s++) st += (s < this.reward.stars ? '★' : '☆');
        UI.text(ctx, st, P.W / 2, y + 116, { size: 20, color: C.sun, align: 'center' });
        UI.text(ctx, '金币 +' + this.reward.coin + (this.reward.wool ? '羊毛 +' + this.reward.wool : ''),
        P.W / 2, y + 142, { size: 12.5, color: C.forest, align: 'center' });
        if (this.newAch && this.newAch.length) {
            UI.text(ctx, '解锁成就: ' + this.newAch[0][1], P.W / 2, y + 162, { size: 11, color: C.soil, align: 'center' });
        }
        } else if (!win) {
            UI.text(ctx, '这一局是有解的，换个取牌顺序再试试', P.W / 2, y + 118, { size: 11.5, color: C.dim, align: 'center' });
        }

        var by = y + 180;
        if (win && this.mode === 'normal') {
            UI.button(ctx, stage, x + 24, by, w - 48, 46, '下一关', function () {
                self.lv = Math.min(CORE.MAX_LEVEL, self.lv + 1); self.seed = null; self.reset();
            }, { bg: C.forest });
        } else if (win) {
            UI.button(ctx, stage, x + 24, by, w - 48, 46, '回到首页', function () { stage.nav('home'); }, { bg: C.forest });
        } else {
            UI.button(ctx, stage, x + 24, by, w - 48, 46, '再试一次', function () { self.seed = null; self.reset(); }, { bg: C.forest });
        }
        UI.button(ctx, stage, x + 24, by + 54, w - 48, 38, self.mode === 'normal' ? '关卡地图' : '回到首页',
        function () { stage.nav(self.mode === 'normal' ? 'levels' : 'home'); }, { bg: C.mint, fg: C.forest });
    };

    if (typeof module !== 'undefined' && module.exports) module.exports = GameScene;
    else if (typeof globalThis !== 'undefined') globalThis.GameScene = GameScene;
})();

/* ===== 微信小游戏版 minigame/js/ui.js ===== */
/* Canvas 绘制与交互基础件。
小游戏没有 DOM，圆角、文本、按钮命中、场景栈这些都得自己实现一遍。
配色与网页版 :root 里的 CSS 变量一一对应。 */
(function () {
    var C = {
        ink: '#264D33', forest: '#2E6B3E', leaf: '#4C9A5A', grass: '#7DBB6E',
        mint: '#D3E8C6', cream: '#F6FAF0', paper: '#FFFFFF', line: '#E2EDD8',
        soil: '#8A6B4A', sun: '#F6C445', dim: '#8AA085', danger: '#B4714E'
    };

    var FONT = 'PingFang SC, Hiragino Sans GB, Microsoft YaHei, sans-serif';

    function font(ctx, size, bold) {
        ctx.font = (bold ? '700 ' : '400 ') + size + 'px ' + FONT;
    }

```

```

function roundRect(ctx, x, y, w, h, r) {
    r = Math.min(r, w / 2, h / 2);
    ctx.beginPath();
    ctx.moveTo(x + r, y);
    ctx.arcTo(x + w, y, x + w, y + h, r);
    ctx.arcTo(x + w, y + h, x, y + h, r);
    ctx.arcTo(x, y + h, x, y, r);
    ctx.arcTo(x, y, x + w, y, r);
    ctx.closePath();
}

function fillRound(ctx, x, y, w, h, r, color) {
    roundRect(ctx, x, y, w, h, r);
    ctx.fillStyle = color;
    ctx.fill();
}

function strokeRound(ctx, x, y, w, h, r, color, lw) {
    roundRect(ctx, x, y, w, h, r);
    ctx.strokeStyle = color;
    ctx.lineWidth = lw || 1.5;
    ctx.stroke();
}

function text(ctx, s, x, y, opt) {
    opt = opt || {};
    font(ctx, opt.size || 14, opt.bold);
    ctx.fillStyle = opt.color || C.ink;
    ctx.textAlign = opt.align || 'left';
    ctx.textBaseline = opt.baseline || 'middle';
    ctx.fillText(s, x, y);
}

/* 简单的中文换行: canvas 没有排版, 只能逐字量宽 */
function wrapText(ctx, s, maxW, size) {
    font(ctx, size || 13, false);
    var out = [], cur = '';
    for (var i = 0; i < s.length; i++) {
        var ch = s[i];
        if (ch === '\n') { out.push(cur); cur = ''; continue; }
        if (ctx.measureText(cur + ch).width > maxW && cur) { out.push(cur); cur = ch; }
        else cur += ch;
    }
    if (cur) out.push(cur);
    return out;
}

/* --- 点击热区: 每帧重建, 命中取最后注册 (最上层) --- */
function HitLayer() {
    this.items = [];
}
HitLayer.prototype.clear = function () { this.items.length = 0; };
HitLayer.prototype.add = function (x, y, w, h, fn, pad) {
    pad = pad || 0;
    this.items.push({ x: x - pad, y: y - pad, w: w + pad * 2, h: h + pad * 2, fn: fn });
};
HitLayer.prototype.hit = function (px, py) {
    for (var i = this.items.length - 1; i >= 0; i--) {
        var it = this.items[i];
    }
}

```

```

        if (px >= it.x && px <= it.x + it.w && py >= it.y && py <= it.y + it.h) return it;
    }
    return null;
};

/* --- 场景栈 --- */
function Stage(plat) {
    this.plat = plat;
    this.ctx = plat.ctx;
    this.hits = new HitLayer();
    this.scene = null;
    this.pending = null;
    this.toastMsg = '';
    this.toastUntil = 0;
    this.now = 0;
    var self = this;
    plat.onTouch({
        start: function (x, y) { self._down = [x, y]; self._downAt = Date.now(); },
        end: function (x, y) {
            if (!self._down) return;
            var dx = x - self._down[0], dy = y - self._down[1];
            self._down = null;
            if (Math.abs(dx) > 12 || Math.abs(dy) > 12) return; // 滑动不算点击
            var h = self.hits.hit(x, y);
            if (h && h.fn) h.fn(x, y);
        },
        move: function (x, y) {
            if (self.scene && self.scene.onMove && self._down) self.scene.onMove(x, y, self._down);
        }
    });
}
Stage.prototype.go = function (scene) { this.pending = scene; };
Stage.prototype.toast = function (msg, ms) {
    this.toastMsg = msg;
    this.toastUntil = Date.now() + (ms || 1600);
};
Stage.prototype.start = function () {
    var self = this;
    var loop = function () {
        self.now = Date.now();
        if (self.pending) {
            if (self.scene && self.scene.exit) self.scene.exit();
            self.scene = self.pending;
            self.pending = null;
            if (self.scene.enter) self.scene.enter(self);
        }
        self.hits.clear();
        var ctx = self.ctx;
        ctx.fillStyle = C.cream;
        ctx.fillRect(0, 0, self.plat.W, self.plat.H);
        if (self.scene) {
            if (self.scene.update) self.scene.update(self.now);
            self.scene.draw(ctx, self);
        }
        self._drawToast(ctx);
        self.plat.raf(loop);
    };
    self.plat.raf(loop);
};
Stage.prototype._drawToast = function (ctx) {

```

```

    if (!this.toastMsg || Date.now() > this.toastUntil) return;
    var w = this.plat.W;
    font(ctx, 12.5, false);
    var tw = ctx.measureText(this.toastMsg).width + 36;
    var x = (w - tw) / 2, y = this.plat.H - 190;
    fillRound(ctx, x, y, tw, 36, 18, 'rgba(38,77,51,0.92)');
    text(ctx, this.toastMsg, w / 2, y + 18, { size: 12.5, color: '#fff', align: 'center' });
};

/* --- 通用按钮 --- */
function button(ctx, stage, x, y, w, h, label, fn, opt) {
    opt = opt || {};
    var bg = opt.bg || C.forest, fg = opt.fg || '#fff';
    fillRound(ctx, x, y, w, h, opt.r == null ? h / 2 : opt.r, bg);
    if (opt.border) strokeRound(ctx, x, y, w, h, opt.r == null ? h / 2 : opt.r, opt.border, 1.5);
;
    text(ctx, label, x + w / 2, y + h / 2, { size: opt.size || 15, bold: true, color: fg, align:
'center' });
    if (fn) stage.hits.add(x, y, w, h, fn, 4);
}

var UI = {
    C: C, FONT: FONT, font: font, roundRect: roundRect, fillRound: fillRound,
    strokeRound: strokeRound, text: text, wrapText: wrapText,
    HitLayer: HitLayer, Stage: Stage, button: button
};

if (typeof module !== 'undefined' && module.exports) module.exports = UI;
else if (typeof globalThis !== 'undefined') globalThis.UI = UI;
})();

/* ===== 微信小游戏版 minigame/js/page.js ===== */
/* 页面外壳：顶部标题、底部导航、可滚动内容区。
   canvas 没有滚动容器，滚动要自己实现：记住偏移量、按内容高度夹住、
   触摸移动时改偏移，并且滚动中不能触发点击。 */
(function () {
    var UI = (typeof require !== 'undefined') ? require('./ui.js') : globalThis.UI;
    var C = UI.C;

    var TABS = [
        { id: 'home', name: '首页', emo: '🏠' },
        { id: 'levels', name: '关卡', emo: '📖' },
        { id: 'rank', name: '排行', emo: '🏆' },
        { id: 'shop', name: '商店', emo: '🛒' },
        { id: 'mine', name: '我的', emo: '🐑' }
    ];
    var TABBAR_H = 62;

    function Page(tab, title) {
        this.tab = tab;
        this.title = title;
        this.scroll = 0;
        this.contentH = 0;
        this._lastY = null;
    }

    Page.prototype.enter = function (stage) {
        this.stage = stage;
        this.scroll = 0;
    }

```

```

    }
    tray.push(t);
    const by={}; tray.forEach(x=>{ (by[x.type]=by[x.type]||[]).push(x); });
    let trio=null; for(const k in by) if(by[k].length>=3){ trio=by[k].slice(0,3); break; }
    if(trio) trio.forEach(x=>{ clearAt.set(x,i); tray.splice(tray.indexOf(x),1); });
  }
  let ok=true;
  rest.concat(held).forEach(t=>{
    if(t.sp!=='bomb') return;
    const c=clearAt.get(t);
    if(c===undefined){ ok=false; return; }
    const start=pos.has(t)?pos.get(t):-1;
    let ins=0; /* 倒数按「之后有几张牌入槽」走，彩虹牌不入槽 */
    for(let i=start+1;i<=c;i++) if(order[i]&&order[i].sp!=='rain') ins++;
    t.bombK=ins+2;
    if(start<0) t.bombLeft=t.bombK; /* 已在卡槽里的炸弹，倒数一并重置 */
  });
  return ok;
}

/* 按全部游戏规则回放一遍，确认这条顺序真的走得通 */
function verifyPlayable(rest,order,held,rainHeld){
  const set=new Set(rest), tray=held.slice(), rainSlot=rainHeld.slice();
  const frozen=new Map(), bomb=new Map();
  rest.forEach(t=>{ if(t.sp==='freeze'&&t.frozen) frozen.set(t,true); });
  tray.forEach(t=>{ if(t.sp==='bomb') bomb.set(t,t.bombLeft|t.bombK); });
  let m=G.matches;
  for(const t of order){
    if(!freeIn(t,set)) return false;
    if(t.sp==='freeze'&&frozen.get(t)) return false;
    if((t.sp==='chain'||t.sp==='supply')&&m<t.chainN) return false;
    set.delete(t);
    for(const o of set) if(o.sp==='freeze'&&frozen.get(o)&&tileNb(o,t)) frozen.set(o,false);
    if(t.sp==='rain'){ rainSlot.push(t); if(rainSlot.length>=3){ rainSlot.length=0; m++; } continue; }
    tray.push(t);
    if(t.sp==='bomb') bomb.set(t,t.bombK);
    let blew=false;
    bomb.forEach((v,k)=>{ if(k!==t&&tray.includes(k)){ bomb.set(k,v-1); if(v-1<=0) blew=true; }
  });
  const by={}; tray.forEach(x=>{ (by[x.type]=by[x.type]||[]).push(x); });
  let trio=null; for(const k in by) if(by[k].length>=3){ trio=by[k].slice(0,3); break; }
  if(trio){ trio.forEach(x=>{ tray.splice(tray.indexOf(x),1); bomb.delete(x); }); m++; }
  else if(blew) return false;
  if(tray.length>=SLOT_N) return false;
}
return set.size===0 && tray.length===0 && rainSlot.length===0;
}

function reshuffleSolvable(){
  const rest=G.tiles.filter(t=>!t.removed);
  if(rest.length<3) return 'empty';
  /* 用本局真正的配置。每日/周常/无尽的 G.lv 都是 1，
  直接 levelDef(G.lv) 会回落到第 1 关，把题面洗成幼儿园难度 */
  const cfg=G.rcfg||{types:levelDef(G.lv).cfg.types,d:levelDef(G.lv).cfg.d};
  const cap=Math.max(3,Math.min(SLOT_N-1,(cfg.d|3)-(toolLv('shuffle')-1)));
  const held=G.tray.filter(x=>!x.matching);
  const rainHeld=G.rain.filter(x=>!x.matching);
  const heldCount={};
  held.forEach(t=>heldCount[t.type]=(heldCount[t.type]||0)+1);

```

```

const plain=rest.filter(t=>t.sp!=='rain');      /* 彩虹牌不参与花色配额 */
const backup=rest.map(t=>t.type);
const bombBackup=rest.concat(held).map(t=>[t,t.bombK,t.bombLeft]);

let needTotal=0;
for(const c in heldCount) needTotal+=3-heldCount[c];
const triples=(plain.length-needTotal)/3;
if(triples<0||triples%1!==0) return 'fail';

for(let attempt=0;attempt<14;attempt++){
  const order=solveOrderLocked(rest,G.matches);
  if(!order) continue;
  const remain={};
  for(const c in heldCount) remain[c]=3-heldCount[c];
  const pool=planColors(cfg.types,triples);
  for(const c in pool) remain[c]=(remain[c]||0)+(pool[c]);
  /* collect 目标: 保证目标花色还留在盘面上, 否则任务被无声废掉。
     搬运量必须是 3 的倍数, 否则会破坏「每色总数是 3 的倍数」这条不变量 */
  const g=G.goal;
  if(g&&g.t==='collect'){
    const want=3*Math.max(1,g.need-g.got);
    if((remain[g.type]||0)<want){
      const donor=Object.keys(remain).map(Number)
        .filter(c=>c!==g.type).sort((a,b)=>remain[b]-remain[a])[0];
      if(donor!=null){
        let move=Math.min(want-(remain[g.type]||0),Math.max(0,(remain[donor]||0)-3));
        move=Math.floor(move/3)*3;
        if(move>0){ remain[donor]-=move; remain[g.type]=(remain[g.type]||0)+move; }
      }
    }
  }
  paintOrder(order,remain,heldCount,cap);
  if(!retimeBombs(rest,order,held,rainHeld)) continue;
  if(!verifyPlayable(rest,order,held,rainHeld)) return 'ok';
}
rest.forEach((t,i)=>{ t.type=backup[i]; }); /* 洗不出可解局面, 原样还回去 */
bombBackup.forEach((t,k,l)=>{ t.bombK=k; t.bombLeft=l; });
return 'fail';
}
function repaint(t){
  t.el.style.setProperty('--cell-c',TYPE_DEFS[t.type].c);
  t.el.innerHTML=face(t.type);
  t.el.style.transform='scale(.72)';
  setTimeout(()=>{ t.el.style.transform=''; },180);
}
function doShuffle(){
  const alive=G.tiles.filter(t=>!t.removed);
  if(alive.length<3){ toast('没有可洗的牌'); SFX.deny(); return false; }
  /* 卡槽已占 6 格且没有任何两张同色 — 下一张必死, 洗牌救不了, 别浪费道具 */
  const hc={};
  G.tray.filter(x=>!x.matching).forEach(t=>hc[t.type]=(hc[t.type]||0)+1);
  if(G.tray.filter(x=>!x.matching).length>=SLOT_N-1 && !Object.keys(hc).some(c=>hc[c]===2)){
    toast('卡槽里没有能凑对的, 洗牌救不了, 试试「移出」');
    SFX.deny(); return false;
  }
}
/* 先洗, 洗不出可解局面就不扣道具 — 绝不能一边说「保证有解」一边给个死局 */
const r=reshuffleSolvable();
if(r!=='ok'){
  toast('这个局面洗不出有解的排布, 道具没有消耗');
  SFX.deny();
}

```

```

    return false;
  }
  if(!spend('shuffle')){ return false; }
  alive.forEach(repaint);
  G.dirty=true;          /* 花色变了，存下来的参考解法不再成立 */
  refreshCover();
  saveProgress();
  SFX.coin();
  toast('已重新洗牌 · 保证仍有解');
  return true;
}
function doUndo(){
  if(![...G.history].reverse().find(t=>t.inTray&&!t.matching)){
    toast('还没有可撤回的操作'); SFX.deny(); return false;
  }
  if(!spend('undo')) return false;
  let n=0;
  for(let i=0;i<toolLv('undo');i++){
    const last=[...G.history].reverse().find(t=>t.inTray&&!t.matching);
    if(!last) break;
    G.tray=G.tray.filter(x=>x!==last);
    G.rain=G.rain.filter(x=>x!==last);
    last.removed=false; last.inTray=false;
    last.el.classList.remove('out');
    G.history=G.history.filter(x=>x!==last);
    n++;
  }
  if(G.goal&&G.goal.t==='steps') G.goal.used=Math.max(0,G.goal.used-n);
  renderTray(); renderRain(); refreshCover(); updateProg(); updateGoal(); saveProgress();
  toast('已撤回 '+n+' 步');
  return true;
}
function doOut(){
  const list=G.tray.filter(x=>!x.matching).slice(0,2+toolLv('out'));
  if(!list.length){ toast('卡槽是空的'); SFX.deny(); return false; }
  if(!spend('out')) return false;
  list.forEach(t=>{
    t.removed=false; t.inTray=false;
    t.el.classList.remove('out');
    G.history=G.history.filter(x=>x!==t);
  });
  G.tray=G.tray.filter(x=>!list.includes(x));
  if(G.goal&&G.goal.t==='steps') G.goal.used=Math.max(0,G.goal.used-list.length);
  renderTray(); refreshCover(); updateProg(); updateGoal(); saveProgress();
  toast('已移出 '+list.length+' 块');
  return true;
}
$('#t-shuffle').onclick=()=>{ if(!G.over) doShuffle(); };
$('#t-undo').onclick=()=>{ if(!G.over) doUndo(); };
$('#t-out').onclick=()=>{ if(!G.over) doOut(); };

/* 失败页复活 */
$('#fail-tips').addEventListener('click',e=>{
  const el=e.target.closest('.tip'); if(!el) return;
  const k=el.dataset.tool;
  if(save.tools[k]<=0){ toast('道具不足，去商店看看吧'); SFX.deny(); return; }
  G.over=false;
  let ok=false;
  if(k==='shuffle'){
    /* 洗牌复活：先把卡槽清空一格再洗，避免立刻又满 */

```

```

        ok=doShuffle();
        if(ok){ const t=G.tray.filter(x=>!x.matching).slice(0,2); t.forEach(x=>{x.removed=false;x.in
Tray=false;x.el.classList.remove('out');G.history=G.history.filter(y=>y!==x);}); G.tray=G.tray.f
ilter(x=>!t.includes(x)); renderTray(); refreshCover(); updateProg(); }
    }
    else if(k==='undo') ok=doUndo();
    else ok=doOut();
    if(ok){
        if(G.deathBy==='time') G.startAt+=30000; /* 超时复活补 30 秒, 否则一回去就又超时 */
        G.deathBy='';
        go('s-game'); requestAnimationFrame(layout); startTimer(); saveProgress();
        toast('复活成功, 继续加油! ');
    }
    else G.over=true;
});
$('#replay-skip').onclick=stopReplay;
$('#fail-replay').onclick=startReplay;
$('#fail-retry').onclick=(()=>{
    clearProgress();
    if(G.mode==='endless'){ G.wave=1; G.score=0; G.buffs=null; }
    go('s-game'); startLevel(G.lv); requestAnimationFrame(layout);
});
$('#win-next').onclick=(()=>{
    clearProgress();
    if(G.mode==='daily' || G.mode==='weekly'){ go('s-game'); startLevel(G.lv); requestAnimationFrame
(layout); return; }
    const nx=G.lv<MAX_LEVEL?G.lv+1:G.lv;
    go('s-game'); startLevel(nx); requestAnimationFrame(layout);
});
/* 暗号分享 */
$('#code-ok').onclick=(()=>$('#codeMask').classList.remove('on'));
/* 点弹层背景关掉, 是通用预期。#buffMask 是必选三选一, 故意不给关 */
['spMask', 'chMask', 'codeMask', 'codeInMask'].forEach(id=>{
    const m=document.getElementById(id);
    if(m) m.addEventListener('click', e=>{ if(e.target===m) m.classList.remove('on'); });
});
$('#btn-code').onclick=(()=>{ $('#code-in').value=''; $('#codeInMask').classList.add('on'); setTi
meout(()=>$('#code-in').focus(), 80); });
$('#code-cancel').onclick=(()=>$('#codeInMask').classList.remove('on'));
$('#code-go').onclick=(()=>{
    const p=parseCode($('#code-in').value);
    if(!p){ toast('暗号看不懂, 检查一下有没有打错'); SFX.deny(); return; }
    $('#codeInMask').classList.remove('on');
    clearProgress(); G.mode='normal'; G.forceSeed=p.seed; G.pendingTarget=p.target||null;
    go('s-game'); startLevel(p.lv); requestAnimationFrame(layout);
    toast(p.target ? ('战书已接下 · 目标 '+fmtTime(p.target.time)) : ('第 '+p.lv+' 关 · 朋友的题面
'));
});
$('#code-in').addEventListener('keydown', e=>{ if(e.key==='Enter') $('#code-go').click(); });
$$('.btn-share').forEach(b=>b.onclick=shareCurrent);

/* 模式入口 */
$('#btn-daily').onclick=(()=>{
    G.mode='daily'; G.wave=1; G.score=0;
    go('s-game'); startLevel(1); requestAnimationFrame(layout);
});
$('#btn-endless').onclick=(()=>{
    G.mode='endless'; G.wave=1; G.score=0; G.buffs=null;
    go('s-game'); startLevel(1); requestAnimationFrame(layout);
});

```

```

$('#btn-weekly').onclick=()=>{
  G.mode='weekly'; G.wave=1; G.score=0;
  go('s-game'); startLevel(1); requestAnimationFrame(layout);
  toast('本周规则: '+weeklyMod().d);
};
$('#btn-restart').onclick=()=>{ clearProgress(); startLevel(G.lv); requestAnimationFrame(layout)
; toast('已重置本关'); };
$('#g-back').onclick=()=>go('s-levels');
$('#btn-sound').onclick=()=>{
  save.sound=!save.sound; persist();
  $('#btn-sound').innerHTML=<i class="fa-solid '+ (save.sound?'fa-volume-high':'fa-volume-xmark'
)+'"></i>;
  applyLang(document.body);
  toast(save.sound?'音效已开启':'音效已关闭');
  if(save.sound) SFX.pick();
};

/* =====
  首页 / 签到
  ===== */
const SAYINGS=[
  '风把云吹散了，剩下的时间，都用来陪你喜欢的羊吧。',
  '慢一点也没关系，草原上的路从来不催人。',
  '今天的第一缕阳光，已经替你晒暖了牧场。',
  '把烦恼交给风，把青草留给羊。',
  '走得远不如走得稳，羊群会一直等你。'
];
function dateKey(d){ d=d||new Date(); return d.getFullYear()+'-'+(d.getMonth()+1)+'-'+d.getDate(
)};
function yesterdayKey(){ const d=new Date(); d.setDate(d.getDate()-1); return dateKey(d); }

function renderHome(){
  const d=new Date();
  const h=d.getHours();
  $('#g-hi').textContent = h<5?'夜深了，牧羊人':h<11?'早上好，牧羊人':h<13?'中午好，牧羊人':h<18
?'下午好，牧羊人':h<22?'傍晚好，牧羊人': '晚安，牧羊人';
  $('#g-date').textContent=(d.getMonth()+1)+'月'+d.getDate()+ '日 · '+['周日','周一','周二','周三
','周四','周五','周六'][d.getDay()];
  $('#g-say').textContent=SAYINGS[(d.getDate()+d.getMonth())%SAYINGS.length];
  /* 用存档里真实的累计数据，不编全服统计 — 本作没有服务端 */
  const done=(save.st&&save.st.matches)||0;
  $('#home-note').textContent = done
  ? '你已经清了 '+fmt(done)+' 组草料，羊群走过 '+save.cleared+' 关'
  : '羊群已经在坡上等你了';
  const p=resumable();
  $('#btn-start').innerHTML=<i class="fa-solid fa-play"></i>'+
  (p ? '继续本局 · 第 '+p.lv+' 关'
  : save.cleared ? '继续闯关 · 第 '+save.level+' 关' : '开始游戏');
  const dk=String(dailySeed());
  const dd=(save.daily||{})[dk];
  $('#daily-sub').textContent = dd ? ('今日 '+fmtTime(dd.time)) : '全服问题';
  $('#btn-daily').classList.toggle('hasdone',!!dd);
  $('#endless-sub').textContent = save.endless ? ('最佳 '+save.endless+' 组') : '最佳 0 组';
  $('#weekly-sub').textContent = weeklyMod().n;
  $('#btn-endless').classList.toggle('hasdone',save.endless>0);
  renderCheckin();
  renderTasks();
  updateChips();
}
function renderCheckin(){

```

```

const ck=save.checkin;
const claimed = ck.date===dateKey();
const box=$('#ck-days'); box.innerHTML='';
const streak=ck.streak||0;
const start=new Date();
start.setDate(start.getDate()-(claimed?Math.max(0,streak-1):streak));
const WD=['周日','周一','周二','周三','周四','周五','周六'];
for(let i=0;i<7;i++){
    const day=new Date(start); day.setDate(start.getDate()+i);
    const doneCount=claimed?streak:streak;
    const isDone=i<doneCount;
    const isToday=(claimed? i===streak-1 : i===streak);
    const wrap=document.createElement('div');
    wrap.className='day';
    /* 奖励直接标在每一格上：原来要签到成功弹个 toast 才知道给了多少，
    看不到「第 7 天有 100」就没有连签的动力。月卡加成也算进去，显示的就是实得数 */
    const base=CK_COIN[i], real=save.pass?base*2+40:base;
    wrap.innerHTML='<div class="dot'+(isToday&&!claimed?' today':(isDone?' done':''))+'>'+
        (isDone&&!isToday&&!claimed)?'<i class="fa-solid fa-check"></i>'+(i+1)+'</div>'+
        '<span'+(isToday?' class="on"':'')+>'+WD[day.getDay()]+</span>'+
        '<span class="rw'+(isToday&&!claimed?' on':'')+(isDone&&!isToday&&!claimed)?' got':'')+>'+
    '<i class="fa-solid fa-circle"></i>'+real+'</span>';
    if(isToday&&!claimed) wrap.onclick=doCheckin;
    else wrap.onclick=(=>toast(claimed?'今天已经签到过啦，明天再来':'先完成前面的签到吧'));
    box.appendChild(wrap);
}
$('#ck-streak').textContent = claimed ? '今日已签 · 连续 '+streak+' 天' : '连续签到 '+streak+'
天 · 点击领取';
}
const CK_COIN=[20,25,30,40,50,60,100];
function doCheckin(){
    const ck=save.checkin;
    if(ck.date===dateKey()){ toast('今天已经签到过啦'); return; }
    ck.streak = (ck.date===yesterdayKey()) ? Math.min(7,(ck.streak||0)+1) : 1;
    ck.date=dateKey();
    let coin=CK_COIN[ck.streak-1];
    if(save.pass) coin=coin*2+40;
    save.coins+=coin;
    let extra='';
    if(ck.streak===7){ save.wool+=20; extra=' + 羊毛 20'; }
    if(save.pass){
        const k=['shuffle','undo','out'][rnd(3)];
        save.tools[k]++; extra+=' + 随机道具 ×1';
    }
    persist(); SFX.coin();
    toast('签到成功! 金币 '+coin+extra);
    renderCheckin(); updateChips();
}

/* =====
    关卡地图
    ===== */
/* 地图最远能翻到哪一章：至少覆盖全部设计关卡，玩家打得更远时跟着延伸 */
function maxMapChapter(){
    return Math.max(Math.ceil(DSIGN_LV/CH_SIZE)-1, chapterIdx(Math.min(save.level,MAX_LEVEL)));
}
let mapCh=null;
function renderLevels(ch){
    if(ch==null) ch = (mapCh!=null?mapCh:chapterIdx(save.level));

```

```

ch=Math.max(0,Math.min(ch,maxMapChapter()));
mapCh=ch;
const first=ch*CH_SIZE+1;
applyChapter(first);
$('#lv-title').textContent=chapterTitle(first);
$('#ch-prev').style.visibility = ch>0?'visible':'hidden';
$('#ch-next').style.visibility = ch<maxMapChapter()?'visible':'hidden';

const map=$('#lv-map'), H=1120;
map.style.height=H+'px';
const pts=[];
for(let i=0;i<CH_SIZE;i++){
  const xp=(i%2===0)?20:60, yp=5+i*10;
  pts.push({lv:first+i,xp,yp,x:xp/100*346,y:yp/100*H});
}
let d='M '+pts[0].x.toFixed(1)+' '+pts[0].y.toFixed(1);
for(let i=1;i<pts.length;i++){
  const a=pts[i-1],b=pts[i];
  const cx=(a.x+b.x)/2+(i%2?76:-76);
  d+=' Q '+cx.toFixed(1)+' '+((a.y+b.y)/2).toFixed(1)+' '+b.x.toFixed(1)+' '+b.y.toFixed(1);
}
map.innerHTML=
  '<svg class="lv-path" viewBox="0 0 346 '+H+'" preserveAspectRatio="none" fill="none">'+
  '<path d="'+d+'" stroke="var(--hill-a)" stroke-width="15" stroke-linecap="round"/>'+
  '<path d="'+d+'" stroke="var(--hill-c)" stroke-width="3.5" stroke-dasharray="1 13" stroke-
linecap="round"/>'+
  '</svg>'+
  '<i class="fa-solid fa-leaf lv-leaf lf-1"></i><i class="fa-solid fa-seedling lv-leaf lf-2"></i>'+
  '<i class="fa-solid fa-leaf lv-leaf lf-3"></i><i class="fa-solid fa-tree lv-leaf lf-4"></i>'+
  '+
  '<i class="fa-solid fa-leaf lv-leaf lf-5"></i>';

pts.forEach(p=>{
  const lv=p.lv, st=save.stars[lv]||0;
  const state = st>0 ? 'done' : (lv===save.level ? 'current' : (lv<save.level ? 'open' : 'lock
'));
  const b=document.createElement('button');
  b.className='lv-node '+state+(isBoss(lv)?' boss':'')+(isBonus(lv)?' bonus':'');
  b.style.left=p.xp+'%'; b.style.top=p.yp+'%';
  let inner = state==='lock' ? '<i class="fa-solid fa-lock"></i>'
    : isBoss(lv) ? '<i class="fa-solid fa-crown"></i>'
    : isBonus(lv) ? '<i class="fa-solid fa-box-open"></i>'
    : st>0 ? '<i class="fa-solid fa-check"></i>' : String(lv);
  if(st>0) inner+=<span class="lv-star">'+[1,2,3].map(k=>'<i class="fa-solid fa-star'+(k<=st?
':' off')+'"></i>').join('')+</span>';
  if(state==='current') inner+=<span class="lv-tag">进行中</span>;
  else if(isBoss(lv)&&state!=='lock') inner+=<span class="lv-tag">BOSS 限时</span>;
  else if(isBonus(lv)&&state!=='lock') inner+=<span class="lv-tag">宝箱关</span>;
  inner+=<span class="lv-label">'+lv+' · '+levelName(lv)+' · '+shapeOf(lv).n+'</span>;
  b.innerHTML=inner;
  b.onclick=()=>{
    if(state==='lock'){ toast('通关第 '+ (lv-1) + ' 关后解锁'); SFX.deny(); return; }
    enterLevel(lv);
  };
  map.appendChild(b);
});

/* 一只羊站在当前进度的节点上，解锁新关时会走过去 */
const curPt=pts.find(p=>p.lv===save.level)||pts.find(p=>!save.stars[p.lv]);

```

```

if(curPt){
  const walker=document.createElement('div');
  walker.className='lv-walker';
  walker.innerHTML='<svg><use href="#sheep"/></svg>';
  const from=pts.find(p=>p.lv===save.level-1);
  const startP=(save.lastWalk!==save.level&&from)?from:curPt;
  walker.style.left=startP.xp+'%'; walker.style.top=(startP.yp)+'%';
  map.appendChild(walker);
  if(startP!==curPt){
    requestAnimationFrame(()=>{
      walker.style.left=curPt.xp+'%'; walker.style.top=curPt.yp+'%';
      walker.classList.add('walking');
    });
    setTimeout(()=>walker.classList.remove('walking'),1300);
  }
  save.lastWalk=save.level; persist();
}

const done=pts.filter(p=>save.stars[p.lv]).length;
const total=Object.keys(save.stars).length;
$('#lv-desc').textContent='本章已通关 '+done+' / '+CH_SIZE+' 关 · 累计 '+total+' 关';
$('#lv-num').textContent=done+'/'+CH_SIZE;
$('#lv-track').style.width=(done/CH_SIZE*100)+'%';
updateChips();
}
$('#rk-seg').addEventListener('click',e=>{
  const b=e.target.closest('button'); if(!b) return;
  rkTab=b.dataset.rk; renderRank();
});
$('#ch-prev').onclick={()=>renderLevels(mapCh-1);}
$('#ch-next').onclick={()=>renderLevels(mapCh+1);}

/* =====
  商店
  ===== */
const PRICE={shuffle:120,undo:80,out:200};
const TILE_SKINS={plain:{n:'清草',cost:0},wood:{n:'木纹',cost:150},cloud:{n:'云朵',cost:260},nigh:
ht:{n:'夜幕',cost:400}};
function renderShop(){
  $$('#s-shop .t-card').forEach(c=>{
    c.querySelector('.have-v').textContent=save.tools[c.dataset.buy];
    c.querySelector('.t-price').lastChild.textContent=Math.round(PRICE[c.dataset.buy]*priceMul()
);
  });
  $('#shop-wool').textContent=fmt(save.wool);
  $('#wallet-wool').textContent=fmt(save.wool);
  /* 道具升级 */
  const up=$('#up-row'); up.innerHTML='';
  Object.keys(TOOL_UP).forEach(k=>{
    const u=TOOL_UP[k], lv=toolLv(k), max=lv>=3, cost=u.cost[lv]||0, poor=save.wool<cost;
    const el=document.createElement('div');
    el.className='up-item';
    el.innerHTML='<div class="u-ico"><i class="fa-solid '+u.ico+'"></i></div>'+
'<div class="u-t"><b>'+u.n+' Lv. '+lv+'</b><span>'+u.desc[lv-1]+'</span>'+
'<div class="u-lv">'+[1,2,3].map(i=>'<i class="'+(i<=lv?'on':'')+'></i>').join('')+</div>'+
'<button class="u-buy'+(max?' max':(poor?' poor':''))+'>'+(max?'已满级':羊毛 '+cost)+'</
button>';
    el.querySelector('.u-buy').onclick={()=>{
      if(max){ toast(u.n+' 已经满级了'); return; }

```

```

        if(save.wool<cost){ toast('羊毛不够, 还差 '+(cost-save.wool)); SFX.deny(); return; }
        save.wool-=cost; save.toolLv[k]=lv+1; persist(); SFX.coin();
        toast(u.n+' 升到 Lv.'+(lv+1)+'': 'u.desc[lv]);
        renderShop();
    };
    up.appendChild(el);
});
/* 牌面皮肤 */
save.tileOwned=save.tileOwned||['plain'];
$$('#skin-row .skin').forEach(el=>{
    const k=el.dataset.tile, own=save.tileOwned.includes(k);
    el.classList.toggle('own',own);
    el.classList.toggle('wear',save.tileSkin===k);
    el.querySelector('.sk-tag').textContent = save.tileSkin===k?'使用中':(own?'点击使用':'羊毛 '
+TILE_SKINS[k].cost);
});
    updateChips();
}
$$('#s-shop .t-card').forEach(c=>{
    c.onclick=()=>{
        const k=c.dataset.buy, p=Math.round(PRICE[k]*priceMul());
        if(save.coins<p){ toast('金币不足, 先去通关攒点吧'); SFX.deny(); return; }
        save.coins-=p; save.tools[k]++; persist(); SFX.coin();
        toast(({shuffle:'洗牌',undo:'撤回',out:'移出'})[k]+' ×1 已放入背包');
        renderShop(); renderTools();
    };
});
$$('#skin-row .skin').forEach(el=>{
    el.onclick=()=>{
        const k=el.dataset.tile;
        save.tileOwned=save.tileOwned||['plain'];
        if(save.tileOwned.includes(k)){
            save.tileSkin=k; persist(); applyTileSkin(); renderShop();
            toast('已换成「'+TILE_SKINS[k].n+'」牌面'); SFX.coin(); return;
        }
        if(save.wool<TILE_SKINS[k].cost){ toast('羊毛不够, 还差 '+(TILE_SKINS[k].cost-save.wool)); S
FX.deny(); return; }
        save.wool-=TILE_SKINS[k].cost; save.tileOwned.push(k); save.tileSkin=k;
        persist(); applyTileSkin(); renderShop(); SFX.coin();
        toast('兑换成功, 牌面换成「'+TILE_SKINS[k].n+'」');
    };
});
function applyTileSkin(){
    const cls='tile-'+(save.tileSkin||'plain');
    [$('#board'),...$$('#queues .grow')].forEach(el=>{
        if(!el) return;
        el.className=el.className.replace(/tile-\w+/g,'').trim()+' '+cls;
    });
}
function applySkin(){
    const s=SHEEP_BY[save.skin];
    const r=document.documentElement.style;
    if(s){
        r.setProperty('--sheep-ink',s.c[0]); r.setProperty('--sheep-wool',s.c[1]);
        r.setProperty('--sheep-face',s.c[2]); r.setProperty('--sheep-blush',s.c[3]);
        r.setProperty('--sheep-ring',s.c[4]);
    }
    document.body.classList.toggle('cb',!!save.cbMode);
}

```

```

/* =====
排行页
-----
全部是真实的本地数据。原来那套地区榜/好友榜是设计稿留下的假数据，
没有服务端就只能是编的，留着只会拉低整页的可信度，已经删掉。
排行的意义建立在「同题可比」上：每日与周常全服同题，暗号让任意两人同题。
===== */
let rkTab='best';
function renderRank(){
  $('#rk-name').textContent=save.name;
  $('#rk-name').setAttribute('data-noi18n',''); /* 昵称是用户数据，三语都原样显示 */
  $('#rk-title').textContent=currentTitle();
  /* 头部三个数字跟着当前面板走。原来固定是「通关/星星/消除组」，
    和下面看的是哪个榜没有关系，而且三个数的口径互相打架：
    cleared 是通关次数（重玩会 +1）却和纪录墙的「N 关有记录」并排，
    matches 又不含无尽。现在每个面板自己带自己的口径 */
  $('#rk-stats').innerHTML=rkStats().map(x=><div><b>'+x[0]+'</b><span>'+x[1]+'</span></div>').join('');
  $('#rk-seg button').forEach(b=>b.classList.toggle('on',b.dataset.rk===rkTab));
  /* 兜底：任何一个面板抛异常都不该让分段控件亮着新标签、内容却停在旧面板上。
    宁可显示一句「读不出来」，也不要一个从此再也点不出来的死标签 */
  let html;
  try{ html=({best:rkBest,daily:rkDaily,endless:rkEndless,duel:rkDuel})[rkTab](); }
  catch(e){ html=rkEmpty('fa-triangle-exclamation','这一栏的记录读不出来<br>存档里的数据可能已损坏'); }
  $('#rk-panel').innerHTML=storageWarn()+html;
  updateChips();
}
/* 每个面板的头部口径：都只统计这个面板真正在比的东西 */
function rkStats(){
  try{
    if(rkTab==='daily'){
      const d=Object.keys(save.daily||{}), w=Object.keys(save.weekly||{});
      const bt=d.map(k=>save.daily[k].time).sort((a,b)=>a-b)[0];
      return [[d.length,'每日次数'],[w.length,'周常周数'],[bt?fmtTime(bt):'-','每日最佳']];
    }
    if(rkTab==='endless'){
      const log=save.endlessLog||[];
      const wave=log.reduce((a,r)=>Math.max(a,r.wave||0),0);
      return [[fmt(save.endless||0),'最佳组数'],[wave||'-','最远波次'],[log.length,'场次']];
    }
    if(rkTab==='duel'){
      const d=save.duel||{win:0,lose:0}, t=d.win+d.lose;
      return [[d.win,'胜'],[d.lose,'负'],[(t?Math.round(d.win/t*100):0)+'%','胜率']];
    }
    const keys=[...new Set([...Object.keys(save.best||{}),...Object.keys(save.stars||{})])].map(
      Number).filter(n=>n>0);
    const three=keys.filter(lv=>(save.stars[lv]||0)>=3).length;
    const ft=keys.map(lv=>(save.best[lv]||{}).time).filter(t=>t>0).sort((a,b)=>a-b)[0];
    return [[keys.length,'有记录'],[three,'三星关'],[ft?fmtTime(ft):'-','最快一关']];
  }catch(e){ return [['-',''],['-',''],['-','']]; }
}
const rkEmpty=(icon,txt)=><div class="rk-card"><div class="rk-empty"><i class="fa-solid '+icon+' "></i>'+txt+'</div></div>;
const rkStars=n=><span class="rk-stars">'+[1,2,3].map(k=><i class="fa-solid fa-star'+(k<=n?'':' off')+' "></i>').join('')+'</span>;

/* 纪录墙：50 关的个人最佳，给已通关的关卡一个重玩理由
  注意取并集：老存档只有 stars 没有 best（用时是后来才开始记的），

```

```

    只看 best 会让「明明通了关却一条纪录都没有」，那是最容易被当成 bug 的空页 */
function rkBest(){
    const src=save.best||{}, st=save.stars||{};
    const keys=[...new Set([...Object.keys(src),...Object.keys(st)])]
        .map(Number).filter(n=>n>0).sort((a,b)=>a-b);
    if(!keys.length) return rkEmpty('fa-medal','还没有任何纪录<br>通关一次就会记下你的用时与最高连
击');
    const timed=keys.filter(lv=>src[lv]&&src[lv].time>0);
    let html='', lastCh=-1;
    keys.forEach(lv=>{
        const ch=chapterIdx(lv);
        if(ch!==lastCh){ lastCh=ch; html+='/div><div class="rk-chapter">'+chapterTitle(lv)+'</div><
div class="rk-card">'; }
        const b=src[lv]||{};
        html+='/div class="rk-row"><span class="n">'+lv+'</span>'+
            '<span class="t">'+levelName(lv)+'</span>'+rkStars(st[lv]||0)+
            '<span class="v">'+(b.time>0?fmtTime(b.time)+'<small>x'+(b.combo||0)+'</small>':<span cla
ss="dim">未记录</span>)+'</span></div>';
    });
    html=html.replace(/^\<\div>/, '')+'</div>';
    let head='<div class="rk-card"><div class="c-hd"><b>个人纪录墙</b><span>'+keys.length+' 关有记
录</span></div>';
    if(timed.length){
        const fast=timed.reduce((a,lv)=>!a||src[lv].time<src[a].time?lv:a,0);
        /* 取全局与墙上行的较大值：老存档里 st.bestCombo 可能比行里的数还小，
        全屏出现「行 x3 / 头部 x0」比数字本身不准更显眼 */
        const cb=timed.reduce((a,lv)=>Math.max(a,src[lv].combo||0),save.st.bestCombo||0);
        head+='/div class="rk-row"><span class="n"><i class="fa-solid fa-bolt"></i></span><span clas
s="t">最快的一关 · '+levelName(fast)+'</span>'+
            '<span class="v">'+fmtTime(src[fast].time)+'</span></div>'+
            '<div class="rk-row"><span class="n"><i class="fa-solid fa-fire"></i></span><span class="t
">历史最高连击</span>'+
            '<span class="v">x'+cb+'</span></div>';
    }else{
        head+='/div class="rk-note">这些关卡是在开始记录用时之前通的，重玩任意一关就会补上成绩</div>
';
    }
    return head+'</div>'+html;
}
/* 存档写不进去时（部分手机浏览器以本地文件打开会禁用 DOM 存储），
成绩只活在这一次会话里。不提示的话，玩家只会觉得「数据莫名其妙没了」 */
function storageWarn(){
    return STORAGE_OK?'':<div class="rk-card"><div class="rk-empty"><i class="fa-solid fa-triangl
e-exclamation"></i>'+
        '当前浏览器禁止本地存档<br>成绩只保留到这个页面关闭为止，换用系统浏览器打开可解决</div></div
>';
}

/* 每日：全服同题，所以「和自己的历史比」是有意义的 */
function rkDaily(){
    const d=save.daily||{};
    const keys=Object.keys(d).sort((a,b)=>b-a);
    if(!keys.length) return rkEmpty('fa-calendar-day','还没打过每日挑战<br>全服同一题面，成绩才有
可比性')+rkWeekly();
    const today=String(dailySeed());
    const mine=d[today];
    const times=keys.map(k=>d[k].time).sort((a,b)=>a-b);
    let head;
    if(mine){
        const rank=times.indexOf(mine.time)+1;

```

```

    head='<div class="rk-big">'+fmtTime(mine.time)+'<small>今日用时</small></div>'+
    '<div class="rk-note">这是你 '+keys.length+' 次每日挑战里第 <b>'+rank+'</b> 快的一次'+
    (rank===1?' — 历史最佳! ':'')+ ' 最高连击 ×'+(mine.combo||0)+'</div>';
  }else{
    head='<div class="rk-big">--:--<small>今日未挑战</small></div>'+
    '<div class="rk-note">今天的题面全服一样，打完就能和自己的历史成绩比一比</div>';
  }
  const best=times[0];
  const rows=keys.slice(0,10).map((k,i)=>{
    const r=d[k], y=k.slice(0,4), m=k.slice(4,6), dd=k.slice(6);
    return '<div class="rk-row'+(r.time===best?' gold':'')+""><span class="n">'+(k===today?'今':
(i+1))+'</span>'+
    '<span class="t">'+(+m)+'月'+(+dd)+'日</span>'+
    '<span class="v">'+fmtTime(r.time)+'<small>×'+(r.combo||0)+'</small></span></div>';
  }).join('');
  return '<div class="rk-card">'+head+'</div>'+
    '<div class="rk-card"><div class="c-hd"><b>最近成绩</b><span>累计 '+keys.length+' 次 · 最佳'+
    '+fmtTime(best)+'</span></div>'+rows+'</div>'+
    rkWeekly();
  }

/* 周常成绩原来只写不读：存了一堆数据，玩家却只能在当次结算页看一眼 */
function rkWeekly(){
  const w=save.weekly||{};
  const keys=Object.keys(w).map(Number).filter(n=>n>0).sort((a,b)=>b-a);
  if(!keys.length) return '';
  const now=weekIndex();
  const rows=keys.slice(0,8).map((k,i)=>{
    const r=w[k], mod=WEEKLY[k%WEEKLY.length];
    return '<div class="rk-row'+(k===now?' gold':'')+""><span class="n">'+(i+1)+'</span>'+
    '<span class="t">'+(k===now?'本周':(now-k)+'周前')+ ' · '+ (mod?mod.n:'周常')+'</span>'+
    '<span class="v">'+fmtTime(r.time)+'<small>×'+(r.combo||0)+'</small></span></div>';
  }).join('');
  return '<div class="rk-card"><div class="c-hd"><b>周常挑战</b><span>共 '+keys.length+' 周</spa
n></div>'+rows+
    '<div class="rk-note">周常每周换一套变体规则，同一周内全服规则相同</div></div>';
  }

/* 无尽：不需要问题就天然可比 */
function rkEndless(){
  const log=save.endlessLog||[];
  /* 判据不能只看 save.endless：首波 0 组失败时它保持 0，
    但 endlessLog 已经记了一条 — 玩家打完看到「还没玩过」，那条记录像是丢了 */
  if(!save.endless&&!log.length) return rkEmpty('fa-infinity','还没玩过无尽草原<br>一波清完接下一波，比谁消得多');
  const rows=log.length? log.map((r,i)=>
    '<div class="rk-row'+(r.score===save.endless?' gold':'')+""><span class="n">'+(i+1)+'</span>'+
    '<span class="t">第 '+r.wave+' 波结束</span>'+
    '<span class="v">'+r.score+'<small>组</small></span></div>').join('')
    : '<div class="rk-empty">还没有历史记录</div>';
  return '<div class="rk-card"><div class="rk-big">'+save.endless+'<small>组 · 历史最佳</small><
/div>'+
    '<div class="rk-note">无尽模式每波都会更难，撑到第几波取决于你的取牌顺序</div></div>'+
    '<div class="rk-card"><div class="c-hd"><b>最近 '+log.length+' 次</b><span>最多保留 10 条</s
pan></div>'+rows+'</div>';
  }

/* 对战：暗号带上成绩就是一封战书，零服务端的异步 PK */
function rkDuel(){

```

```

const d=save.duel||{win:0,lose:0,log:[]};
const total=d.win+d.lose;
const rate=total?Math.round(d.win/total*100):0;
const rows=(d.log||[]).length? d.log.map(r=>
  '<div class="rk-row "+(r.win?'win':'lose')+""><span class="n">'+(r.win?'胜':'负')+</span>'+
  '<span class="t">第 '+r.lv+ ' 关 · '+levelName(r.lv)+</span>'+
  '<span class="v">'+fmtTime(r.mine)+<small>对 '+fmtTime(r.theirs)+</small></span></div>').j
oin('')
  : '<div class="rk-empty">还没有对战记录</div>';
/* 胜负胜率已经在头部卡片里，这里不再重复一遍 */
return '<div class="rk-card"><div class="c-hd"><b>怎么发起对战</b>'+
  (total?'<span>共 '+total+ ' 场 · 胜率 '+rate+'%</span>':''+</div>'+
  '<div class="rk-note">通关后在结算页点「把这局的暗号分享给朋友」，暗号会<b>带上你的成绩</b>
。'+
  '对方在首页输入这串暗号，会拿到一模一样的盘面，并且开局就看得要追的目标 — 通关时自动判胜
负。<br>'+
  '整个过程不需要联网，成绩就编码在那串字符里。<br>'+
  '<b>只有通关的对局才计入胜负</b>：没打过去、或中途重开新盘面，都不算负，所以上面的胜率天然偏
高。</div></div>'+
  '<div class="rk-card"><div class="c-hd"><b>最近对战</b><span>最多保留 20 条</span></div>'+ro
ws+</div>';
}

/* =====
图鉴 / 牧场 / 成就
===== */
function renderDex(){
  const own=Object.keys(save.dex||{});
  $('#dex-count').textContent=own.length+ ' / '+SHEEP.length;
  renderDexEx(own.length);
  const cur=activeSheep();
  $('#dex-cur').innerHTML = cur
  ? '<svg style="'+sheepStyle(cur)+'"><use href="#sheep"/></svg>'+
    '<div class="c-info"><b>出战: '+cur.n+ ' · Lv.'+sheepLv(cur.id)+</b>'+
    '<span>'+PASSIVE_TXT[cur.p.t](Math.round(cur.p.v*sheepMul(cur.id)))+</span>'+
    '<span class="c-story">「'+cur.s+」</span></div>'
  : '<svg><use href="#sheep"/></svg><div class="c-info"><b>还没有出战的羊</b><span>通关有几率
遇到新羊，点下面的头像即可出战</span></div>';
  const g=$('#dex-grid'); g.innerHTML='';
  SHEEP.forEach(s=>{
    const has=!(save.dex&&save.dex[s.id]);
    const lv=sheepLv(s.id);
    const d=document.createElement('div');
    d.className='dex-cell'+(has?'':' lock')+(save.skin===s.id&&has?' on':'');
    d.innerHTML='<span class="rr r'+s.r+'">'+RARITY[s.r]+</span>'+
      '<svg style="'+sheepStyle(s)+'"><use href="#sheep"/></svg>'+
      '<b>'+(has?s.n:'? ? ?')+</b>'+
      '<span class="lvtag">'+(has?('Lv.'+lv+ ' · '+PASSIVE_TXT[s.p.t](Math.round(s.p.v*sheepMul(s
.id)))):'未收集')+</span>'+
      '<span class="shard">'+(has&&save.dex[s.id].s?'碎片 '+save.dex[s.id].s:'')+</span>';
    d.onclick=()=>{
      if(!has){ toast('还没遇到这只羊，多通几关试试'); SFX.deny(); return; }
      save.skin=s.id; persist(); applySkin(); renderDex(); SFX.coin();
      toast(s.n+'：'+s.s);
    };
    g.appendChild(d);
  });
  updateChips();
}
/* =====

```

玩法说明

特殊牌原本只有「第一次遇到时弹一次」的教学（#spMask），弹完就再也找不回来，而炸弹倒数、锁链组数这类规则恰恰是要回头查的。这里把牌面、道具、关卡目标、四种模式集中成一页常驻手册，文案直接取自 SPECIALS / TOOL_UP，不另抄一份，免得规则改了说明还停在旧版

```
===== */
const HELP_BASE=[
  ['fa-layer-group','怎么算赢','点击盘面上没被压住的牌，它会飞进下方卡槽；卡槽里凑齐 3 张同花色自动消除。把盘面、卡槽、彩虹槽全部清空就算过关。'],
  ['fa-table-columns','卡槽只有 7 格','卡槽塞满且没有任何三连时判负。这是唯一的失败方式（限时与限步关卡另算）。周常变体会把格子改成 5 格或 9 格。'],
  ['fa-box-open','底部牌堆','盘面下方的牌堆只能取最前面一张；少数关卡的牌堆是双向的，两端都能取。'],
  ['fa-clover','必定有解','每一局都是先算出一条合法的通关顺序，再沿着它铺花色和特殊牌——有解是构造出来的，不是碰运气。卡在死局只会是取牌顺序的问题，失败页可以直接看参考解法回放。']
];
const HELP_MODES=[
  /* 章节数按 CHAPTERS.length 说：50 关 ÷ CH_SIZE(10) = 5 章，原来写的「四个章节」
   与 CHAPTERS 里实际的五个章节对不上。宝箱关 isBonus 排除了前 12 关，第一章没有 */
  ['fa-map','闯关','50 关，五个章节。每章第 10 关是 BOSS 关；第二章起每章第 5 关是宝箱关，金币翻倍、必掉图鉴。'],
  ['fa-calendar-day','每日挑战','全服同一套题面，每天换一次。因为同题，成绩才有可比性。'],
  ['fa-calendar-week','周常挑战','每周换一套变体规则（窄槽、迷雾、爆破等），同一周内全服相同。'],
  ['fa-infinity','无尽草原','一波清完接下一波，每波更难，中途可选增益。比的是累计消除组数。']
];
function renderHelp(){
  const box=$('#help-body'); if(!box) return;
  const card=(title,rows)=><div class="hp-card"><div class="hp-hd">'+title+'</div>'+rows+'</div>';
  const row=(ico,name,tip,extra)=><div class="hp-row"><div class="hp-ico"><i class="fa-solid '+ico+'"></i></div>'+
    '<div class="hp-t"><b>'+name+'</b><span>'+tip+'</span></div>'+(extra||'')+</div>';

  let html=card('基本规则', HELP_BASE.map(r=>row(r[0],r[1],r[2])).join(''));

  /* 规则牌与奖励牌分开：前者会改变消除顺序，后者只在被消掉时给东西 */
  const rule=['freeze','chain','dbl','rain','bomb','mystery','supply'];
  const reward=['coin','wool','gift','torch'];
  const seen=save.seenSp||[];
  const spRow=k=>{
    const s=SPECIALS[k];
    return row(s.icon, s.name, s.tip, seen.includes(k)?<span class="hp-tag">已遇到</span>:'');
  };
  html+=card('规则牌 <span>会改变取牌顺序</span>', rule.map(spRow).join(''));
  html+=card('奖励牌 <span>只在被消掉时给东西</span>', reward.map(spRow).join(''));

  html+=card('道具 <span>可在商店升级</span>', Object.keys(TOOL_UP).map(k=>{
    const t=TOOL_UP[k], lv=toolLv(k);
    return row(t.ico, t.n+' · Lv.'+lv, t.desc[lv-1]+'。'+
      (k==='shuffle'?洗出来的一定还是有解的局面，洗不出就不消耗道具。':
      k==='undo'?把最近入槽的牌退回原位，炸弹倒数也会跟着重置。':
      '把卡槽里的牌临时挪出去，给三连腾地方。'),
      lv<3?'<span class="hp-tag dim">可升级</span>':<span class="hp-tag">满级</span>');
  })).join(''));

  html+=card('关卡目标 <span>闯关模式</span>',
    [['fa-check','清空全部','把盘面、卡槽、彩虹槽全部清空'],
    ['fa-hashtag','限步','在限定步数内消够指定组数'],
```

```

    ['fa-bullseye','指定花色','消掉指定组数的某一种花色'],
    ['fa-layer-group','剥层','清掉最上面四层的指定张数'],
    ['fa-paw','救援','把被压在最底下的小羊挖出来'],
    ['fa-hourglass-half','限时','在限定秒数内消够指定组数']].map(r=>row(r[0],r[1],r[2])).join('
'));

html+=card('模式', HELP_MODES.map(r=>row(r[0],r[1],r[2])).join(''));
/* 第三方署名: 图标字体是 CC BY 4.0, 许可要求署名, 且要放在用户看得到的地方 */
html+=card('关于', row('fa-circle-info','羊羊羊之草原牧歌 V1.0','单文件网页实现, 全部资源内嵌
, 运行期间不请求任何外部资源。')+
    row('fa-tag','图标字体 Font Awesome 6 Free','Copyright (c) Fonticons, Inc. 字体依 SIL OFL 1
.1、图标依 CC BY 4.0 授权, 本软件仅嵌入使用, 未修改其字形。'));
    box.innerHTML=html;
}

/* 碎片兑换 + 保底进度 + 全收集奖励, 都放在图鉴顶部 */
function renderDexEx(ownN){
    const box=$('#dex-ex'); if(!box) return;
    if(dexFull()){
        box.className='dex-ex full';
        box.innerHTML='<div class="ex-hd"><b><i class="fa-solid fa-crown"></i> 图鉴全满</b>'+
            '<span>金币与羊毛产出 +5% (已生效)</span></div>';
        return;
    }
    box.className='dex-ex';
    const sp=surplusShards(), left=Math.max(0,PITY_N-(save.pity||0));
    const btns=[1,2,3].map(r=>{
        const fresh=SHEEP.filter(s=>s.r===r&&!(save.dex||{})[s.id]).length;
        const cost=EXCHANGE[r], can=fresh&&sp>=cost;
        return '<button class="ex-b'+(can?'':' off')+' " data-ex="'+r+'">'+
            '<b>'+RARITY[r]+'</b><span>'+(fresh?cost+' 片':'已集齐')+'</span></button>';
    }).join('');
    box.innerHTML='<div class="ex-hd"><b><i class="fa-solid fa-puzzle-piece"></i> 富余碎片 '+sp+'<
/b>'+
        '<span>满级羊多出来的碎片可以换缺的羊</span></div><div class="ex-row">'+btns+'</div>'+
        '<div class="ex-note">再有 '+left+' 次重复掉落必出新羊</div>';
    box.querySelectorAll('[data-ex]').forEach(b=>b.onclick=()=>exchangeSheep(+b.dataset.ex));
}

function renderFarm(){
    $('#farm-wool').textContent=fmt(save.wool);
    const total=Object.values(save.farm||{}).reduce((a,b)=>a+b,0);
    $('#farm-title').textContent = total>=20?'丰饶牧场' : total>=12?'像样的牧场' : total>=5?'初具
规模' : '草原一角';
    $('#farm-sub').textContent='建设等级合计 '+total+' / 20 · 金币 '+(+farmLv('grass')*2)+'% · 羊
毛 '+(+farmLv('mill')*2)+'%';
    const list=$('#farm-list'); list.innerHTML='';
    FARM.forEach(f=>{
        const lv=farmLv(f.id), max=lv>=f.max, cost=f.cost(lv), poor=save.wool<cost;
        const el=document.createElement('div');
        el.className='farm-item';
        el.innerHTML='<div class="f-ico"><i class="fa-solid '+f.ico+' "></i></div>'+
            '<div class="f-info"><b>'+f.n+' Lv. '+lv+'</b><span>'+f.desc(lv)+(max?'':' → '+f.desc(lv+1)
)+'</span>'+
            '<div class="f-bar">'+Array.from({length:f.max},(_,i)=>'<i class="'+(i<lv?'on':'')+' "></i>
').join('')+'</div></div>'+
            '<button class="f-buy'+(max?' max':'(poor?' poor':''))+' ">'+(max?'已满':'升级')+
            (max?'':'<small>羊毛 '+cost+'</small>')+'</button>';
        el.querySelector('.f-buy').onclick=()=>{
            if(max){ toast(f.n+' 已经建满了'); return; }
            if(save.wool<cost){ toast('羊毛不够, 还差 '+(+cost-save.wool)); SFX.deny(); return; }
        }
    });
}

```

```

        save.wool-=cost; save.farm[f.id]=lv+1; persist(); SFX.coin();
        toast(f.n+' 升到 Lv.'+(lv+1)+'! '+f.desc(lv+1));
        checkAch(); renderFarm();
    });
    list.appendChild(el);
});
updateChips();
}
function renderAch(){
    save.ach=save.ach||[];
    $('#ach-count').textContent=save.ach.length+' / '+ACH.length;
    const list=$('#ach-list'); list.innerHTML='';
    ACH.forEach(a=>{
        const got=save.ach.includes(a[0]);
        const el=document.createElement('div');
        el.className='ach-item'+(got?' got':'');
        el.innerHTML='<div class="a-ico"><i class="fa-solid '+ (got?'fa-medal':'fa-lock')+' "></i></div>'+
v>'+
        '<div class="a-t"><b>'+a[1]</b><span>'+a[2]</span></div>'+
        '<span class="a-w">羊毛 '+a[4]</span>';
        list.appendChild(el);
    });
    updateChips();
}

/* =====
我的
===== */
function renderMine(){
    /* 语言胶囊：文案本身不参与翻译（各语言都显示自己的名字），所以直接写死 */
    const lp=$('#lang-pick');
    if(lp){
        lp.innerHTML=LANGS.map(([k,label])=>
            '<b data-lang="'+k+'"' + ((save.lang||'zh')===k?' class="on"':'')+'>'+label+'</b>').join('');
    };
}

$('#me-name').textContent=save.name;
$('#me-name').setAttribute('data-noi18n',''); /* 昵称是用户数据，任何语言下都原样显示 */
$('#me-lv').textContent='青草认证牧羊人 · Lv.'+Math.min(99,1+Math.floor(save.cleared/2));
$('#me-title').textContent=currentTitle();
$('#st-coin').textContent=fmt(save.coins);
$('#st-wool').textContent=fmt(save.wool);
$('#st-lv').textContent=save.level;
$('#badge-sub').textContent='已解锁 '+ (save.ach||[]).length+' / '+ACH.length+' 枚';
$('#badge-n').textContent=(save.ach||[]).length;
$('#bag-sub').textContent='已收集 '+Object.keys(save.dex||{}).length+' / '+SHEEP.length+
    (activeSheep()?(' · 出战 '+activeSheep().n): '');
$('#farm-row-sub').textContent='建设等级 '+Object.values(save.farm||{}).reduce((a,b)=>a+b,0)+'
/ 20';
document.addEventListener('click',e=>{
    const b=e.target.closest('#lang-pick b');
    if(!b) return;
    setLang(b.dataset.lang);
    SFX.tap&&SFX.tap();
});
$$('#s-mine [data-toggle]').forEach(row=>{
    row.querySelector('.switch').classList.toggle('on',!!save[row.dataset.toggle]);
});
updateChips();

```

```

}
$$('#s-mine [data-toggle]').forEach(row=>{
  row.onclick={()=>{
    const k=row.dataset.toggle;
    save[k]=!save[k]; persist();
    row.querySelector('.switch').classList.toggle('on',save[k]);
    if(k==='sound'){ $('#btn-sound').innerHTML='<i class="fa-solid '+ (save.sound?'fa-volume-high
': 'fa-volume-xmark') +' "></i>'; if(save.sound) SFX.pick(); }
    if(k==='vibrate'&&save.vibrate) buzz(30);
    if(k==='cbMode') applySkin();
    if(k==='bgm'){ save.bgm?BGM.start():BGM.stop(); }
    toast(({sound:'音效',vibrate:'震动反馈',bgm:'背景音乐',cbMode:'色觉辅助'})[k]+(save[k]?' 已
开启':' 已关闭'));
  }
});
$('#me-edit').onclick={()=>{
  const n=prompt('给自己取个名字吧',save.name);
  if(n&&n.trim()){ save.name=n.trim().slice(0,10); persist(); renderMine(); toast('名字已更新');
}
});
$('#row-badge').onclick={()=>go('s-ach');
$('#row-bag').onclick={()=>go('s-dex');
$('#row-farm').onclick={()=>go('s-farm');
$('#row-help').onclick={()=>go('s-help');
$('#me-title').onclick={()=>{
  const ok=TITLES.filter(titleUnlocked);
  const i=ok.findIndex(t=>t[1]===currentTitle());
  const next=ok[(i+1)%ok.length];
  save.title=next[1]; persist(); renderMine();
  toast(ok.length>1?('称号已换成「'+next[1]+'」'):'再解锁一些成就就能换称号');
};
$('#row-cloud').onclick={()=>{ persist(); toast('进度已保存到本机浏览器'); }};
$('#row-about').onclick={()=>toast('羊羊羊之草原牧歌 V1.0');
let resetArm=0;
$('#row-reset').onclick={()=>{
  const now=Date.now();
  if(now-resetArm<3000){
    localStorage.removeItem(SAVE_KEY);
    save=load(); applySkin(); renderMine(); updateChips();
    toast('进度已清空, 重新开始牧羊生活');
    resetArm=0;
  }else{
    resetArm=now;
    toast('再点一次确认清空全部进度');
  }
};

/* =====
阶段五: 新手引导
前三关分步教学, 每一步只在真正的时机出现, 讲完就不再打扰
===== */
const GUIDE=[
  {at:'start1', txt:'点一下最上面那张牌, 它就会飞进下面的卡槽。被别的牌压住的会变灰, 点不动。'},
  {at:'pick1', txt:'看下面这排格子 — 这就是卡槽, 一共 7 格。凑齐 3 张一样的就会自动消掉。'},
  {at:'match1', txt:'漂亮! 3 秒内连着消就能攒连击, 连击越高金币越多。'},
  {at:'half', txt:'卡槽快满时会变黄变红。7 格全被占满又消不掉, 这一关就输了。'},
  {at:'tools', txt:'过不去的时候用下面三个道具: 洗牌打乱、撤回退一步、移出退回 3 张。'}
];
function guideStep(key){
  if(save.guide>=GUIDE.length) return;

```

```

if(cur!=='s-game') return;          /* 引导只属于牌局这一屏 */
if(G.mode!=='normal' || G.lv>3) return;
const idx=GUIDE.findIndex(g=>g.at===key);
if(idx<0 || idx!==save.guide) return;
const c=$('#coach');
$('#coach-txt').textContent=GUIDE[idx].txt;
relang(c);
c.classList.add('on');
const t0=Date.now();
$('#coach-ok').onclick=(=>{
  c.classList.remove('on');
  save.guide=idx+1; persist();
  G.startAt+=Date.now()-t0;          /* 引导期间不计时 */
});
}

/* =====
阶段五：程序化 BGM
没有音频素材，用 WebAudio 现场合成一段五声音阶循环。
真要上线还是应该换成正经的音乐资源，这里只是把「有背景音乐」这件事跑通
===== */
const BGM={
  timer:0, step:0,
  scale:[392.00,440.00,493.88,587.33,659.25,783.99],
  start(){
    if(this.timer||!save.bgm) return;
    const c=ac(); if(!c) return;
    this.timer=setInterval(()=>this.tick(),480);
  },
  stop(){ clearInterval(this.timer); this.timer=0; },
  tick(){
    /* 页面不可见时 AudioContext 会挂起，继续建振荡器只会白白堆积 */
    if(!save.bgm||document.hidden||cur!=='s-game'){ this.stop(); return; }
    const c=ac(); if(!c) return;
    const s=this.scale;
    const n=s[(this.step*3+((this.step/4|0)%2?2:0))%s.length];
    this.note(n/2,.9,.030);
    if(this.step%2===0) this.note(n,1.1,.016);
    if(this.step%8===0) this.note(s[0]/4,1.8,.030);
    this.step++;
  },
  note(f,dur,vol){
    const c=ac(); if(!c) return;
    const t0=c.currentTime;
    const o=c.createOscillator(), g=c.createGain(), lp=c.createBiquadFilter();
    lp.type='lowpass'; lp.frequency.value=1400;
    o.type='sine'; o.frequency.setValueAtTime(f,t0);
    g.gain.setValueAtTime(0,t0);
    g.gain.linearRampToValueAtTime(vol,t0+.08);
    g.gain.exponentialRampToValueAtTime(.0001,t0+dur);
    o.connect(lp); lp.connect(g); g.connect(c.destination);
    o.start(t0); o.stop(t0+dur+.05);
  }
};

/* =====
阶段四：养成与收集
-----
红线：所有养成加成必须以「经济向」为主，涉及玩法的效果只放在极少数羊上，
并且开满之后要用模拟器重跑定标曲线，各关通过率上浮不得超过 8 个点。

```

前面花了很大力气把难度做成可控的，不能在这里被养成数值冲掉。

```

===== */

/* --- 羊图鉴：24 只，配色复用同一个 SVG，美术成本为零 --- */
const SHEEP=[
  {id:'hat', n:'草帽羊', r:1, p:{t:'coin',v:3}, c:['#2E6B3E','#FFFFFF','#FBFDF7','#F6C445',
  ,'#D3E8C6'], s:'第一只跟着你出门的羊，草帽是它自己叼来的。'},
  {id:'moss', n:'苔藓羊', r:1, p:{t:'wool',v:3}, c:['#3D6B4A','#EFF6EC','#F8FCF6','#A8C98F',
  ,'#CFE3C6'], s:'喜欢在背阴的石头上打盹，毛里总有青苔味。'},
  {id:'cloud', n:'云朵羊', r:1, p:{t:'coin',v:3}, c:['#5A7A8C','#FBFDF7','#FFFFFF','#BBD5E4',
  ,'#D8E7F0'], s:'走路很轻，远看像一朵挪动的云。'},
  {id:'wheat', n:'麦穗羊', r:1, p:{t:'wool',v:3}, c:['#8A6B2A','#FFF9E8','#FFFD4', '#E8C878',
  ,'#EDE0BE'], s:'秋天最忙，据说能背回半亩地的麦穗。'},
  {id:'mint', n:'薄荷羊', r:1, p:{t:'coin',v:4}, c:['#3E8574','#F0FBF7','#F8FEFC','#9FD9C6',
  ,'#CEEDE3'], s:'路过的地方都会留下一股清凉气。'},
  {id:'clay', n:'陶土羊', r:1, p:{t:'wool',v:4}, c:['#8A5A3E','#FFF3EA','#FFFAF5','#DBA07A',
  ,'#EBD3C2'], s:'在河滩长大，蹄子上永远沾着泥。'},
  {id:'dew', n:'露珠羊', r:1, p:{t:'coin',v:4}, c:['#3C7E93','#F0AFD','#F9FDF', '#9AD2E4',
  ,'#CDE9F2'], s:'天没亮就出门，只为舔到最新鲜的那滴露水。'},
  {id:'straw', n:'干草羊', r:1, p:{t:'wool',v:4}, c:['#9A7B36','#FFFBEE','#FFFD7', '#E0C489',
  ,'#EEE2C4'], s:'冬天的粮仓管理员，睡觉也抱着草垛。'},
  {id:'fern', n:'蕨叶羊', r:1, p:{t:'coin',v:4}, c:['#2F6B4E','#EEF8F1','#F7FCF9','#8CC9A4',
  ,'#C9E6D5'], s:'认得林子里每一株蕨，从不迷路。'},
  {id:'stone', n:'碎石羊', r:1, p:{t:'wool',v:4}, c:['#5E6670','#F5F6F8','#FBFBFC','#B6BEC8',
  ,'#DCE0E6'], s:'沉默寡言，撞过的石头比走过的路多。'},
  {id:'peach', n:'蜜桃羊', r:1, p:{t:'coin',v:5}, c:['#A85A5A','#FFF2F2','#FFF9F9','#F0A9A9',
  ,'#F3D6D6'], s:'夏天会偷偷去果园，回来时脸是红的。'},
  {id:'lilac', n:'紫丁香', r:1, p:{t:'wool',v:5}, c:['#6E5A96','#F7F2FD','#FCFAFF','#C0A8E0',
  ,'#DED2F0'], s:'黄昏时最好看，毛尖泛着淡淡的紫。'},
  {id:'olive', n:'橄榄羊', r:1, p:{t:'coin',v:5}, c:['#6B7B36','#F7FAEC','#FCFDF6','#BCCB84',
  ,'#DDE6C2'], s:'不挑食，据说吃过牧场里所有能吃的东西。'},
  {id:'coral', n:'珊瑚羊', r:1, p:{t:'wool',v:5}, c:['#B06248','#FFF4F0','#FFFAF8','#F0AE94',
  ,'#F5D9CE'], s:'从很远的海边走来，还记得潮水的声音。'},

  {id:'scarf', n:'围巾羊', r:2, p:{t:'combo',v:600}, c:['#2F6C8F','#F4FAFE','#FBFDF', '#EE9A8',
  ,'#CDE4F2'], s:'围巾是牧羊人织的，它一年四季都不肯摘。'},
  {id:'snow', n:'初雪羊', r:2, p:{t:'combo',v:600}, c:['#5C7A96','#FFFFFF','#FBFDF', '#B9D4E',
  ,'#DCEAF4'], s:'落雪那天出生，从此不怕冷。'},
  {id:'amber', n:'琥珀羊', r:2, p:{t:'coin',v:7}, c:['#9A6516','#FFF6E4','#FFFCF2','#E8B34',
  ,'#F0DDB0'], s:'毛色像凝固的阳光，靠近会觉得暖。'},
  {id:'ivy', n:'常春羊', r:2, p:{t:'wool',v:7}, c:['#2C6A3C','#EDF7EE','#F7FCF8','#82C08',
  ,'#C4E2CA'], s:'爬过最高的墙，也蹭掉过最多的毛。'},
  {id:'thaw', n:'暖阳羊', r:2, p:{t:'thaw',v:2}, c:['#B07A1E','#FFF8E6','#FFFD4', '#F2C55',
  ,'#F2E3B8'], s:'走到哪儿，哪儿的冰就化了。'},
  {id:'peek', n:'千里羊', r:2, p:{t:'peek',v:2}, c:['#456B9A','#F2F7FE','#FAFCFF','#A8C3E',
  ,'#D3E1F5'], s:'站在坡顶能看清三里外的草，眼神好得离谱。'},
  {id:'lucky', n:'四叶羊', r:2, p:{t:'drop',v:30}, c:['#3A8A50','#EFAF1','#F8FDF9','#8FD3A',
  ,'#C8EAD2'], s:'总能踩到四叶草，连它自己都觉得奇怪。'},

  {id:'star', n:'星月羊', r:3, p:{t:'luck',v:1}, c:['#4B3A78','#F5F1FD','#FCFAFF','#C9A6F',
  ,'#DED4F2'], s:'夜里会抬头很久，像在等谁。'},
  {id:'aurora', n:'极光羊', r:3, p:{t:'coin',v:10}, c:['#2F7A86','#EEFBFB','#F8FEFE','#7ED8D',
  ,'#C2ECEC'], s:'极夜里见过天上的光，回来后毛就变了颜色。'},
  {id:'golden', n:'鎏金羊', r:3, p:{t:'wool',v:10}, c:['#8A6A12','#FFFAE6','#FFFD3', '#F2CE5',
  ,'#F0E2B4'], s:'传说中被阳光亲吻过的羊，很少有人真的见过。'}
];
const SHEEP_BY={}; SHEEP.forEach(s=>SHEEP_BY[s.id]=s);
const RARITY=['','普通','稀有','传说'];
const PASSIVE_TXT={
  coin:v=>'金币产出 '+v+'%',

```

```

    wool:v=>'羊毛产出 '+v+'%',
    combo:v=>'连击窗口 '+v+(v/1000).toFixed(1)+' 秒',
    thaw:v=>'开局自动解冻 '+v+' 张冰冻牌',
    peek:v=>'开局自动翻开 '+v+' 张谜之牌',
    drop:v=>'图鉴掉落率 '+v+'%',
    luck:v=>'通关必定掉落一只羊'
  });
  /* 羊等级: 重复掉落攒碎片, 3 / 8 片升一级, 效果 x1.5 / x2 */
  const SHARD_NEED=[0,3,8];
  const SHARD_MAX=8;      /* 超过这个数的碎片对本羊已无用, 可以拿去兑换缺的羊 */
  const PITY_N=12;        /* 连续 12 次掉落都是重复, 下一次必给新羊 */
  const EXCHANGE={1:6, 2:12, 3:25}; /* 兑换一只未收集的羊要多少富余碎片 */
  function surplusShards(){
    return Object.keys(save.dex||{}).reduce((a,id)=>a+Math.max(0,((save.dex[id]||{}).s||0)-SHARD_MAX),0);
  }
  function dexFull(){ return Object.keys(save.dex||{}).length>=SHEEP.length; }
  /* 兑换: 只花「已经没用的」碎片, 从富余最多的羊身上先扣,
    所以不会把还在升级路上的羊拆掉 */
  function exchangeSheep(r){
    const fresh=SHEEP.filter(s=>s.r===r&&!(save.dex||{})[s.id]);
    if(!fresh.length){ toast('这个稀有度已经集齐了'); SFX.deny(); return; }
    const cost=EXCHANGE[r], have=surplusShards();
    if(have<cost){ toast('富余碎片不够, 还差 '+cost-have+' 片'); SFX.deny(); return; }
    let need=cost;
    Object.keys(save.dex).sort((a,b)=>(save.dex[b].s||0)-(save.dex[a].s||0)).forEach(id=>{
      if(need<=0) return;
      const take=Math.min(Math.max(0,(save.dex[id].s||0)-SHARD_MAX),need);
      save.dex[id].s-=take; need-=take;
    });
    const pick=fresh[Math.floor(Math.random()*fresh.length)];
    save.dex[pick.id]={s:0}; save.pity=0;
    persist(); checkAch(); renderDex(); SFX.coin();
    toast('碎片换来了 '+pick.n+' : '+pick.s);
  }
  function sheepLv(id){
    const sh=(save.dex||{})[id]; if(!sh) return 0;
    return sh.s>=SHARD_NEED[2]?3 : sh.s>=SHARD_NEED[1]?2 : 1;
  }
  function sheepMul(id){ return [0,1,1.5,2][sheepLv(id)]||0; }
  function activeSheep(){ return (save.dex&&save.dex[save.skin])?SHEEP_BY[save.skin]:null; }
  function passive(type){
    const s=activeSheep();
    if(!s||s.p.t!==type) return 0;
    return s.p.v*sheepMul(s.id);
  }
}

/* --- 牧场: 4 项建设, 纯经济加成, 完全不碰难度 --- */
const FARM=[
  {id:'grass', n:'青草地', ico:'fa-seedling', max:5, cost:l=>60+l*40, desc:l=>'金币产出 '+l*(2)+'%'},
  {id:'mill', n:'风车', ico:'fa-fan', max:5, cost:l=>80+l*50, desc:l=>'羊毛产出 '+l*(2)+'%'},
  {id:'fence', n:'围栏', ico:'fa-border-all',max:5, cost:l=>70+l*45, desc:l=>'商店道具便宜 '+l*(3)+'%'},
  {id:'pen', n:'羊圈', ico:'fa-house-chimney', max:5, cost:l=>90+l*60, desc:l=>'图鉴掉落率 '+l*(4)+'%'}
];
function farmLv(id){ return (save.farm||{})[id]||0; }
/* 图鉴全满: +5% 金币与羊毛。只碰经济不碰难度,

```

```

    养成加成不得让通过率上浮超过 8 个点这条红线仍然成立 */
function dexFullBonus(){ return dexFull()?0.05:0; }
function coinMul(){ return 1 + farmLv('grass')*0.02 + passive('coin')/100 + dexFullBonus(); }
function woolMul(){ return 1 + farmLv('mill')*0.02 + passive('wool')/100 + dexFullBonus(); }
function priceMul(){ return 1 - farmLv('fence')*0.03; }
function dropRate(){ return 0.34 + farmLv('pen')*0.04 + passive('drop')/100; }

/* --- 道具等级: 用难度旋钮做洗牌升级, 洗出来的局面必胜路线更宽 --- */
const TOOL_UP={
  shuffle:{n:'洗牌',ico:'fa-shuffle', cost:[0,120,260], desc:['有解洗牌','有解洗牌 · 局面更宽松','有解洗牌 · 洗成最宽松局面']},
  undo: {n:'撤回',ico:'fa-rotate-left',cost:[0,100,220], desc:['撤回 1 步','连续撤回 2 步','连续撤回 3 步']},
  out: {n:'移出',ico:'fa-arrow-right-from-bracket',cost:[0,140,300], desc:['移出 3 张','移出 4 张','移出 5 张']},
};
function toolLv(k){ return (save.toolLv||{})[k]||1; }

/* --- 成就: 36 条, 全部由真实统计推出 --- */
const ACH=[
  ['first','初次放羊','通关第 1 关',s=>s.cleared>=1,20],
  ['c5','小有所成','累计通关 5 关',s=>s.cleared>=5,20],
  ['c15','熟能生巧','累计通关 15 关',s=>s.cleared>=15,30],
  ['c40','牧场老手','累计通关 40 关',s=>s.cleared>=40,50],
  ['c80','草原传说','累计通关 80 关',s=>s.cleared>=80,80],
  ['lv5','越走越远','解锁到第 5 关',s=>s.level>=5,20],
  ['lv12','出了新手村','解锁到第 12 关',s=>s.level>=12,30],
  ['lv21','翻过雪线','解锁到第 21 关',s=>s.level>=21,50],
  ['lv31','走到星夜','解锁到第 31 关',s=>s.level>=31,80],
  ['st10','攒星星','累计 10 颗星',s=>starTotal()>=10,20],
  ['st30','三十而立','累计 30 颗星',s=>starTotal()>=30,40],
  ['st60','星光满地','累计 60 颗星',s=>starTotal()>=60,70],
  ['st99','星河灿烂','累计 99 颗星',s=>starTotal()>=99,120],
  ['perfect','一尘不染','有 1 关拿到三星',s=>Object.values(s.stars||{}).some(v=>v>=3),25],
  ['perfect5','五星连珠','有 5 关拿到三星',s=>Object.values(s.stars||{}).filter(v=>v>=3).length>=5,45],
  ['perfect15','完美主义','有 15 关拿到三星',s=>Object.values(s.stars||{}).filter(v=>v>=3).length>=15,90],
  ['combo5','小连击','单局连击达到 5',s=>(s.st.bestCombo||0)>=5,20],
  ['combo10','连击好手','单局连击达到 10',s=>(s.st.bestCombo||0)>=10,35],
  ['combo20','连击大师','单局连击达到 20',s=>(s.st.bestCombo||0)>=20,70],
  ['m100','消了一百组','累计消除 100 组',s=>(s.st.matches||0)>=100,20],
  ['m500','消了五百组','累计消除 500 组',s=>(s.st.matches||0)>=500,40],
  ['m2000','消除机器','累计消除 2000 组',s=>(s.st.matches||0)>=2000,90],
  ['boss1','初见 Boss','通关 1 个 Boss 关',s=>(s.st.boss||0)>=1,40],
  ['boss3','Boss 克星','通关 3 个 Boss 关',s=>(s.st.boss||0)>=3,80],
  ['notool','徒手过关','不用任何道具通关 1 次',s=>(s.st.noTool||0)>=1,25],
  ['notool10','老手风范','不用道具通关 10 次',s=>(s.st.noTool||0)>=10,60],
  ['daily1','今日打卡','完成 1 次每日挑战',s=>(s.st.daily||0)>=1,25],
  ['daily7','坚持一周','完成 7 次每日挑战',s=>(s.st.daily||0)>=7,70],
  ['endless20','无尽入门','无尽模式消 20 组',s=>(s.endless||0)>=20,25],
  ['endless60','无尽好手','无尽模式消 60 组',s=>(s.endless||0)>=60,60],
  ['dex5','小小收藏家','收集 5 只羊',s=>Object.keys(s.dex||{}).length>=5,30],
  ['dex12','半个图鉴','收集 12 只羊',s=>Object.keys(s.dex||{}).length>=12,60],
  ['dex24','图鉴全满','收集全部 24 只羊',s=>Object.keys(s.dex||{}).length>=24,150],
  ['sr','传说降临','收集到 1 只传说羊',s=>Object.keys(s.dex||{}).some(id=>SHEEP_BY[id]&&SHEEP_BY[id].r===3),80],
  ['farm5','初具规模','牧场建设累计 5 级',s=>Object.values(s.farm||{}).reduce((a,b)=>a+b,0)>=5,40],
  ['farm20','牧场主','牧场全部建满',s=>Object.values(s.farm||{}).reduce((a,b)=>a+b,0)>=20,120]
];

```

```

];
/* 称号：由成就解锁，可在「我的」页切换 */
const TITLES=[
  ['', '牧羊新手', ()=>true],
  ['c15', '熟练牧人', null],
  ['c40', '牧场老手', null],
  ['c80', '草原传说', null],
  ['perfect15', '完美主义者', null],
  ['combo20', '连击大师', null],
  ['boss3', 'Boss 克星', null],
  ['daily7', '风雨无阻', null],
  ['dex24', '图鉴收藏家', null],
  ['sr', '传说饲主', null],
  ['farm20', '牧场主', null],
  ['st99', '星河灿烂', null]
];
function titleUnlocked(t){ return !t[0] || (save.ach||[]).includes(t[0]); }
function currentTitle(){
  const t=TITLES.find(x=>x[1]===save.title && titleUnlocked(x));
  return t?t[1]:'牧羊新手';
}
function starTotal(){ return Object.values(save.stars||{}).reduce((a,b)=>a+b,0); }
function achDone(a){ try{ return !!a[3](save); }catch(e){ return false; } }
function checkAch(){
  save.ach=save.ach||[];
  let got=[];
  ACH.forEach(a=>{
    if(save.ach.includes(a[0])) return;
    if(achDone(a)){ save.ach.push(a[0]); save.wool+=a[4]; got.push(a); }
  });
  if(got.length){
    persist();
    setTimeout(()=>toast('成就达成: '+got[0][1]+(got.length>1?' 等 '+got.length+' 项:')+'. 羊毛 '+got.reduce((s,a)=>s+a[4],0)),500);
  }
  return got;
}

function comboWindow(){ return COMBO_WINDOW+passive('combo')+((G.buffs&&G.buffs.combo)||0); }
/* 开局生效的被动：解冻 / 翻牌。都只作用于特殊牌，幅度很小 */
function applyStartPassives(){
  G.aidUsed=false;
  /* 续局时不能再执行一遍，否则反复退出重进能把整盘冰冻牌全化开 */
  if(G.passivesDone) return;
  G.passivesDone=true;
  const th=passive('thaw');
  if(th){
    G.tiles.filter(t=>t.sp==='freeze'&&t.frozen).slice(0,Math.round(th)).forEach(t=>{
      t.frozen=false; drawTile(t);
    });
  }
  const pk=passive('peek');
  if(pk){
    G.tiles.filter(t=>t.sp==='mystery'&&!t.shown).slice(0,Math.round(pk)).forEach(t=>{
      t.shown=true; drawTile(t);
    });
  }
}
/* 结算页展示掉落的羊 */
function showDrop(d){

```

```

const box=$('#win-drop');
if(!d){ box.classList.remove('on'); box.innerHTML=''; return; }
const s=d.sheep;
box.classList.add('on');
box.innerHTML='<div class="dp-sheep" style="'+sheepStyle(s)+'"><svg><use href="#sheep"/></svg>
</div>'+
  '<div class="dp-info"><b>'+s.n+'<span class="dp-r r'+s.r+'">'+RARITY[s.r]+'</span></b>'+
  '<span>'+(d.dup?'已有，转为碎片 x1（当前 '+save.dex[s.id].s+'） ':'新羊入队！'+PASSIVE_TXT[s.
p.t](s.p.v))+'</span></div>';
}
function sheepStyle(s){
  return '--sheep-ink: '+s.c[0]+';--sheep-wool: '+s.c[1]+';--sheep-face: '+s.c[2]+'
  ';--sheep-blush: '+s.c[3]+';--sheep-ring: '+s.c[4];
}

/* =====
每日任务
-----
纯留存钩子：只发资源，不给任何玩法加成，所以完全不碰难度红线。
每天用日期播种抽三条，进度按自然日重置。
===== */
const TASKS=[
  {id:'match', stat:'matches', goals:[20,30,45], txt:g=>'累计消除 '+g+' 组',      coin:70, wool
:14},
  {id:'clear', stat:'clears', goals:[1,2,3], txt:g=>'通关 '+g+' 关',      coin:90, woo
l:18},
  {id:'combo', stat:'combo', goals:[4,5,6], txt:g=>'单局达成 '+g+' 连击',      coin:80, woo
l:16, max:true},
  {id:'play', stat:'plays', goals:[3,5,7], txt:g=>'开局 '+g+' 次',      coin:55, woo
l:11},
  {id:'spec', stat:'spec', goals:[6,10,15], txt:g=>'消掉 '+g+' 张特殊牌',      coin:85, woo
l:17},
  {id:'daily', stat:'dailyRun',goals:[1], txt:()=>>'完成一次每日挑战',      coin:110, wo
ol:22}
];
const TASK_BY={}; TASKS.forEach(t=>TASK_BY[t.id]=t);
function ensureTasks(){
  const key=dateKey();
  if(!save.tasks||save.tasks.date!==key){
    /* 用日期播种抽三条，同一天进游戏拿到的任务是固定的 */
    const r=mulberry32(dailySeed());
    const pool=TASKS.slice();
    for(let i=pool.length-1;i>0;i--){ const j=Math.floor(r()*(i+1)); [pool[i],pool[j]]=pool[j],
pool[i]}; }
  save.tasks={ date:key, prog:{}, list:pool.slice(0,3).map(t=>({
    id:t.id, goal:t.goals[Math.floor(r()*t.goals.length)], claimed:false
  })));
  persist();
}
return save.tasks;
}
function bumpTask(stat,val){
  const T=ensureTasks();
  const def=TASKS.find(t=>t.stat===stat);
  T.prog[stat] = def&&def.max ? Math.max(T.prog[stat]||0,val) : (T.prog[stat]||0)+val;
  persist();
}
function taskGot(it){ return (ensureTasks().prog[TASK_BY[it.id].stat])||0; }
function taskDone(it){ return taskGot(it)>=it.goal; }
function claimTask(it){

```

```

    if(it.claimed||!taskDone(it)) return false;
    const d=TASK_BY[it.id];
    it.claimed=true;
    save.coins+=Math.round(d.coin*coinMul());
    save.wool+=Math.round(d.wool*woolMul());
    persist(); SFX.coin(); updateChips();
    toast('任务完成! 金币 '+Math.round(d.coin*coinMul())+' · 羊毛 '+Math.round(d.wool*woolMul())
);
    return true;
}
function renderTasks(){
    const T=ensureTasks();
    const box=$('#task-list'); if(!box) return;
    const left=T.list.filter(it=>!it.claimed).length;
    $('#task-sub').textContent = left?('还有 '+left+' 项没领'):'今日已全部完成';
    box.innerHTML='';
    T.list.forEach(it=>{
        const d=TASK_BY[it.id], got=Math.min(taskGot(it),it.goal), done=taskDone(it);
        const el=document.createElement('div');
        el.className='task-row'+(it.claimed?' claimed':'(done?' ready':''));
        el.innerHTML='<div class="tk-t"><b>'+d.txt(it.goal)+'</b>'+
            '<div class="tk-bar"><i style="width: '+got/it.goal*100)+'%"></i></div></div>'+
            '<div class="tk-r"><span>'+got+'/'+it.goal+'</span>'+
            '<button class="tk-btn">'+(it.claimed?'已领':(done?'领取':'金币 '+d.coin))+'</button></div>
        >';
        el.querySelector('.tk-btn').onclick=()=>{
            if(it.claimed){ toast('这项已经领过了'); return; }
            if(!done){ toast('还差 '+it.goal-got+' 就能领了'); SFX.deny(); return; }
            claimTask(it); renderTasks();
        };
        box.appendChild(el);
    });
}

/* =====
    无尽模式：波间三选一增益
    全部是「放宽约束」类型（多格子 / 少花色 / 少块数 / 送道具），
    所以下一波的构造依然成立，有解性不受影响
    ===== */
const BUFFS=[
    {id:'slot', ico:'fa-table-columns', n:'多一格卡槽', d:'接下来每一波都多一个格子，效果会叠加'}
,
    {id:'color', ico:'fa-palette', n:'少一种花色', d:'下一波开始花色少一种，配对更容易'},
    {id:'small', ico:'fa-compress', n:'盘面缩小', d:'下一波少 12 张牌，跑得更快'},
    {id:'tool', ico:'fa-toolbox', n:'补给两件', d:'立刻获得 2 个随机道具'},
    {id:'combo', ico:'fa-fire', n:'连击更宽松', d:'连击窗口延长 1.5 秒，更容易连起来'}
];
function offerBuff(){
    const r=shuffle(BUFFS.slice()).slice(0,3);
    $('#buff-title').textContent='第 '+G.wave+' 波来了';
    const g=$('#buff-grid'); g.innerHTML='';
    r.forEach(b=>{
        const el=document.createElement('button');
        el.className='buff-opt';
        el.innerHTML='<div class="bf-ico"><i class="fa-solid '+b.ico+'"></i></div>'+
            '<div class="bf-t"><b>'+b.n+'</b><span>'+b.d+'</span></div>';
        el.onclick=()=>{ applyBuff(b.id); $('#buffMask').classList.remove('on');
            startLevel(1); requestAnimationFrame(layout); };
        g.appendChild(el);
    });
}

```

```

    relang('#buffMask');
    $('#buffMask').classList.add('on');
}
function applyBuff(id){
    G.buffs=G.buffs||{slots:0,types:0,tiles:0,combo:0};
    if(id==='slot') G.buffs.slots++;
    if(id==='color') G.buffs.types++;
    if(id==='small') G.buffs.tiles+=12;
    if(id==='combo') G.buffs.combo+=1500;
    if(id==='tool'){ for(let i=0;i<2;i++) save.tools[TOOL_KEYS[rnd(3)]]++; persist(); renderTools(
    ); }
    SFX.coin();
}

/* --- 掉落 --- */
function rollDrop(lv){
    if(!passive('luck') && Math.random()>dropRate()) return null;
    save.dex=save.dex||{};
    const rnd=a=>a[Math.floor(Math.random()*a.length)];
    const roll=Math.random();
    const want = roll<0.06?3 : roll<0.28?2 : 1;
    const fresh=SHEEP.filter(s=>!save.dex[s.id]);
    let pick;
    if(fresh.length&&(save.pity||0)>=PITY_N){
        /* 保底：优先补抽中档位里缺的，那一档满了就补最低稀有度里缺的 ——
           保底不该成为白嫖传说的通道 */
        const inTier=fresh.filter(s=>s.r===want);
        const lowest=Math.min.apply(null,fresh.map(s=>s.r));
        pick=rnd(inTier.length?inTier:fresh.filter(s=>s.r===lowest));
    }else{
        /* 平时也向「缺的」倾斜：稀有度先掷，档内七成概率优先给没有的那只。
           否则后期几乎全是重复，最后一只纯靠运气 */
        const pool=SHEEP.filter(s=>s.r===want), fp=pool.filter(s=>!save.dex[s.id]);
        pick=(fp.length&&Math.random()<0.7)?rnd(fp):rnd(pool);
    }
    if(!pick) return null;
    if(save.dex[pick.id]){ save.dex[pick.id].s++; save.pity=(save.pity||0)+1; return {sheep:pick,d
up:true}; }
    save.dex[pick.id]={s:0}; save.pity=0;
    return {sheep:pick,dup:false};
}

/* =====
   公共 UI
   ===== */
function updateChips(){
    $$('.coin-v').forEach(el=>el.textContent=fmt(save.coins));
    /* 这个函数在 applyLang 之后还会改写文本，不补一次翻译就会漏 */
    if(typeof applyLang==='function' && (save.lang||'zh')!=='zh') applyLang(document.querySelector
('section.on'));
}

/* 首页开始按钮：有未打完的一局就直接续上 */
$('#btn-start').onclick=(=>{
    /* 按钮上写着「第 N 关」，点下去就该直接进第 N 关。
       原来没有存档时会跳到关卡地图，和按钮上的承诺对不上；
       想挑关卡的话，底部导航栏第二个就是地图 */
    const p=resumable();
    enterLevel(p ? p.lv : Math.max(1,save.level||1));
});

```

```

/* 切后台 / 关页面前保住当前进度 */
document.addEventListener('visibilitychange',()=>{
  if(document.hidden){
    FX.clear(); BGM.stop();
    G.hiddenAt=Date.now(); /* 切后台不该继续烧限时关的时间 */
    if(cur==='s-game') saveProgress();
  }else{
    if(G.hiddenAt&&cur==='s-game'&&!G.over){ G.startAt+=Date.now()-G.hiddenAt; }
    G.hiddenAt=0;
    if(cur==='s-game'&&save.bgm) BGM.start();
  }
});
addEventListener('pagehide',()=>{ if(cur==='s-game') saveProgress(); });

/* 尺寸变化重排 */
let rzTimer;
window.addEventListener('resize',()=>{
  clearTimeout(rzTimer);
  rzTimer=setTimeout(()=>{ if(cur==='s-game'&&G.tiles.length) layout(); },120);
});

/* 键盘快捷键（桌面调试友好） */
document.addEventListener('keydown',e=>{
  if(cur!=='s-game') return;
  if(e.key==='r') startLevel(G.lv);
  if(e.key==='1') doShuffle();
  if(e.key==='2') doUndo();
  if(e.key==='3') doOut();
});

/* 首次交互解锁音频 */
document.addEventListener('pointerdown',()=>ac(),{once:true});

/* ===== 启动 ===== */
/* 页面级渲染函数跑完自动补一次翻译。逐个手工补必然漏 ——
  这轮已经漏过 updateChips、对局页标题、失败页三处。
  对局内的高频渲染（renderTray/renderTools）不包，避免每次点牌都遍历 DOM，
  它们由 startLevel 与 renderGoal 处的显式 relang 覆盖。 */
['renderHome','renderLevels','renderShop','renderRank','renderDex','renderHelp',
'renderFarm','renderAch','renderMine','renderTasks','renderCheckin','renderDexEx',
'renderGoal','renderFailTips'].forEach(n=>{
  const f = window[n];
  if(typeof f !== 'function') return;
  window[n] = function(){ const r = f.apply(this, arguments); relang(); return r; };
});

function boot(){
  detectFA();
  applyLang(document.body); /* 启动页与标题也归语言管，别等到进首页才翻 */
  syncDocTitle();
  FX.init();
  applySkin();
  updateChips();
  renderTools();
  $('#btn-sound').innerHTML=<i class="fa-solid '+ (save.sound?'fa-volume-high':'fa-volume-xmark'
)+'"></i>;
  setTimeout(()=>{ if(cur==='s-splash') go('s-home'); },2200);
  $('#s-splash').onclick=()=>{ if(cur==='s-splash') go('s-home'); }; /* 允许跳过 */
}
boot();

```

```

window.addEventListener('load',()=>{
    const before=FA_OK;
    detectFA();
    if(before!=FA_OK && cur==='s-game') buildBoard();
});
</script>
</body>
</html>

```

```

/* ===== 安卓版 android/java/com/cityg/caoyuan/MainActivity.java ===== */
package com.cityg.caoyuan;

import android.app.Activity;
import android.content.ClipData;
import android.content.ClipboardManager;
import android.content.Context;
import android.graphics.Color;
import android.os.Bundle;
import android.view.View;
import android.view.WindowManager;
import android.webkit.JavascriptInterface;
import android.webkit.ValueCallback;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;

/**
 * 只是一个装游戏的壳：一个全屏 WebView 加载 assets 里的单文件游戏。
 * 没有网络权限，所有资源都在包内。
 */
public class MainActivity extends Activity {

    private WebView web;

    @Override
    protected void onCreate(Bundle state) {
        super.onCreate(state);

        getWindow().addFlags(WindowManager.LayoutParams.FLAG_DRAWS_SYSTEM_BAR_BACKGROUNDS);
        getWindow().setStatusBarColor(Color.parseColor("#F6FAF0"));
        getWindow().setNavigationBarColor(Color.parseColor("#FFFFFF"));
        // 状态栏是浅色底，图标要转成深色，否则白字看不见
        View decor = getWindow().getDecorView();
        decor.setSystemUiVisibility(decor.getSystemUiVisibility()
            | View.SYSTEM_UI_FLAG_LIGHT_STATUS_BAR);

        web = new WebView(this);
        WebSettings s = web.getSettings();
        s.setJavaScriptEnabled(true);
        // 存档全靠 localStorage，这一项不开的话进度一关就没
        s.setDomStorageEnabled(true);
        s.setDatabaseEnabled(true);
        s.setAllowFileAccess(true);
        s.setCacheMode(WebSettings.LOAD_NO_CACHE);
        // 音效与背景音乐是合成音，不该要求先点一下才能响
        s.setMediaPlaybackRequiresUserGesture(false);
        // 不跟随系统字体缩放，否则版式会被撑坏
        s.setTextZoom(100);
        s.setSupportZoom(false);

```

```

        s.setBuiltInZoomControls(false);

        web.addJavascriptInterface(new Clip(), "AndroidClip");
        web.setWebViewClient(new WebViewClient() {
            @Override public void onPageFinished(WebView v, String url) {
                // file:// 是不安全来源, navigator.clipboard 根本不存在,
                // 暗号「复制」会静默失败。这里补一个同名实现, 页面代码不用改
                v.evaluateJavascript(
                    "(function(){ if(!navigator.clipboard && window.AndroidClip){\"
                    + \" navigator.clipboard={writeText:function(t){\"
                    + \"     try{ AndroidClip.set(String(t)); return Promise.resolve(); }\"
                    + \"     catch(e){ return Promise.reject(e); } }};\"
                    + \"})();\"), null);
            }
        });
        web.setBackgroundColor(Color.parseColor("#F6FAF0"));
        web.setOverScrollMode(View.OVER_SCROLL_NEVER);
        web.setLongClickable(false);

        setContentView(web);
        web.loadUrl("file:///android_asset/index.html");
    }

    /**
     * 游戏是单页应用, 没有浏览器历史可退。返回键交给页面判断:
     * 不在首页就回首页 (在局中先存进度), 已在首页才退出。
     */
    @Override
    public void onBackPressed() {
        if (web == null) { super.onBackPressed(); return; }
        web.evaluateJavascript(
            "(function(){try{\"
            + \" if(typeof cur!=='undefined'&&cur!=='s-home'){\"
            + \"     if(cur==='s-game'&&typeof saveProgress==='function') saveProgress();\"
            + \"     go('s-home'); return 'stay';\"
            + \" }\"
            + \"}catch(e){} return 'exit';})();\",
            new ValueCallback<String>() {
                @Override public void onReceiveValue(String v) {
                    if (v == null || v.contains("exit")) finish();
                }
            }
        ));
    }

    /** 切后台时让页面自己存一次盘, 防止被系统杀掉丢进度 */
    @Override
    protected void onPause() {
        super.onPause();
        if (web != null) {
            web.evaluateJavascript(
                \"try{ if(typeof cur!=='undefined'&&cur==='s-game'&&typeof saveProgress==='functi
on') saveProgress(); }catch(e){}\",
                null);
            web.onPause();
        }
    }

    @Override
    protected void onResume() {
        super.onResume();
    }

```

```

        if (web != null) web.onResume();
    }

    /** 只暴露一个写剪贴板的方法，不给页面任何其它系统能力 */
    private class Clip {
        @JavascriptInterface
        public void set(String text) {
            ClipboardManager cm =
                (ClipboardManager) getSystemService(Context.CLIPBOARD_SERVICE);
            if (cm != null) cm.setPrimaryClip(ClipData.newPlainText("暗号", text));
        }
    }
}

/* ===== 安卓版 android/AndroidManifest.xml ===== */
<?xml version="1.0" encoding="utf-8"?>
<!-- 没有 INTERNET 权限：这个游戏完全离线，装完不联网也能玩 -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.cityg.caoyuan"
    android:versionCode="115"
    android:versionName="1.0">

    <uses-sdk android:minSdkVersion="21" android:targetSdkVersion="34" />

    <application
        android:label="@string/app_name"
        android:icon="@drawable/ic_launcher"
        android:roundIcon="@drawable/ic_launcher"
        android:theme="@android:style/Theme.Material.Light.NoActionBar"
        android:hardwareAccelerated="true"
        android:allowBackup="true">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait"
            android:configChanges="orientation|screenSize|keyboardHidden|uiMode|smallestScreenSize|s
creenLayout">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>

/* ===== 安卓版 android/res/values/strings.xml ===== */
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">羊羊羊之草原牧歌</string>
</resources>

/* ===== 安卓版 android/res/drawable/ic_launcher.xml ===== */
<?xml version="1.0" encoding="utf-8"?>
<!-- 用矢量图当图标：省掉一整套 mipmap 位图，也不需要图像工具 -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp" android:height="108dp"

```

```

        android:viewportWidth="108" android:viewportHeight="108">

<!-- 底色：草原绿 -->
<path android:fillColor="#2E6B3E"
      android:pathData="M0,0h108v108h-108z" />

<!-- 远山 -->
<path android:fillColor="#3C7C4C"
      android:pathData="M0,78 C18,62 34,74 54,66 C74,58 90,72 108,62 L108,108 L0,108 Z" />

<!-- 羊身：三团白云状 -->
<path android:fillColor="#FFFFFF"
      android:pathData="M36,52 m-13,0 a13,13 0 1,0 26,0 a13,13 0 1,0 -26,0z" />
<path android:fillColor="#FFFFFF"
      android:pathData="M54,44 m-14,0 a14,14 0 1,0 28,0 a14,14 0 1,0 -28,0z" />
<path android:fillColor="#FFFFFF"
      android:pathData="M72,52 m-13,0 a13,13 0 1,0 26,0 a13,13 0 1,0 -26,0z" />
<path android:fillColor="#FFFFFF"
      android:pathData="M54,60 m-16,0 a16,16 0 1,0 32,0 a16,16 0 1,0 -32,0z" />

<!-- 羊脸 -->
<path android:fillColor="#EAF3E4"
      android:pathData="M54,66 m-9,0 a9,10 0 1,0 18,0 a9,10 0 1,0 -18,0z" />
<!-- 眼睛 -->
<path android:fillColor="#2E6B3E"
      android:pathData="M50,64 m-1.6,0 a1.6,1.6 0 1,0 3.2,0 a1.6,1.6 0 1,0 -3.2,0z" />
<path android:fillColor="#2E6B3E"
      android:pathData="M58,64 m-1.6,0 a1.6,1.6 0 1,0 3.2,0 a1.6,1.6 0 1,0 -3.2,0z" />
</vector>

```